CONTENTS

Overview: Vision, Scope, Actors, and Fulfillment Pipelines

Title: Product Overview and Strategic Direction
Date: 2026-07-03
Status: Draft-for-owner-review
Source: Master Plan 006 overview, module reports, and BA requirements from specs 001–014.

---

1. Product Vision

Mission Statement:
Consolidate fragmented Etsy order management, design workflow, and fulfillment operations into a unified Odoo 19 CE platform that:

1. **Ingest Etsy orders** via API (primary) and email (fallback)
2. **Route fulfillment** to three pipelines: internal VN production, Gearment POD dropship, or hybrid
3. **Manage design workflow** from order → mockup approval → production files → pickup/drop-ship
4. **Track shipments** via GKE Excel + GDrive integrations and push tracking back to Etsy
5. **Control product catalog** via Excel bulk sync, multichannel listing logic, and Odoo-to-Etsy publish (Phase 3)

Timeframe: Phases 0–5 (2026-05 through 2026-10+). Phase 1 (order ingest + tracking + dashboards) live on staging 2026-07-03; Phase 3 (catalog publish) begins 2026-07-04.

---

2. Key Actors & Roles

USER ROLES (5 PRIMARY)

Role	Org Unit	Key Responsibilities	Module Group	ACL Gate
Operations Manager	Fulfillment	Assign orders to internal/dropship, approve address changes, monitor tracking, manage inventory sync	OPS (dashboards, approvals, GKE tracking)	`group_multichannel_ops_lead`
Production Lead (VN)	Production	Manage design queue, mark orders ready-for-design, schedule picking, print batches, track in-house fulfillment	PRODUCTION (design file routing, fulfillment state, picking)	`group_vn_production_lead`
Gearment Operator	Fulfillment	Generate Gearment quotes, accept POes, monitor dropship status, reconcile shipping	DROPSHIP (GKE, Gearment API, tracking)	`group_gearment_dropship`
Marketing / Etsy Manager	Sales	Configure Etsy shop, publish listings, manage shipping profiles, monitor order dashboard	ETSY + CATALOG (shop config, listing publish, currency)	`group_etsy_shop_manager`
BA/Product Lead	Product	Create design rules, manage product catalog, configure category routing, set pricing overrides, generate Excel exports	CATALOG + OPS (multi-design rules, SKU derivation, export wizard)	`group_product_lead`

BA USER GROUPS (7 BACKEND GROUPS PER MODULE)

Group	Purpose	Module	Models	Permissions
`group_etsy_shop_manager`	Etsy shop OAuth, config, order view	etsy_integration	etsy.shop, sale.order	CREATE, WRITE (shop level), READ (orders by shop_id)
`group_multichannel_ops_lead`	Ops dashboard, order approvals, sync health	multichannel_hub_core	sale.order, sale.order.fulfillment, multichannel.sync.health	WRITE (approval, state transitions), READ (all)
`group_vn_production_lead`	Design file routing, production state	multichannel_hub_core	design.file, design.file.route, order.pipeline.state	CREATE (routes), WRITE (design approval, state), READ (all)
`group_gearment_dropship`	Gearment API access, quote/PO, tracking	multichannel_hub_fulfillment	purchase.order (dropship PO + quote fields), sale.order.fulfillment	READ/WRITE (quote + PO), READ (tracking)
`group_product_lead`	Product hub, catalog sync, listing override, publish	multichannel_hub_core	multichannel.listing, product.product, product_template_attribute_value	WRITE (listing override, publish), CREATE (catalog via import)
`group_multichannel_syncer`	Automated syncs (cron-triggered)	multichannel_hub_core, etsy_integration	multichannel.sync.health, etsy.email.log, etsy.api.log, gearment.api.log	WRITE (health flag, log entry), READ (all)
`group_analytics_reader`	Dashboard-only read access	multichannel_hub_core	sale.order, sale.order.fulfillment (computed fields only)	READ (all dashboards, no CREATE/WRITE)

---

3. Scope Boundaries

IN SCOPE (PHASES 0–3)

- **Order Ingest** — Etsy API v3 (primary, Phase 1) and email (fallback, Phase 0 legacy)
- **Order Routing** — Fulfillment pipeline assignment by product category + design complexity
- **Design Workflow** — File upload, approval, production file routing, GDrive integration
- **Tracking Import** — GKE Excel + GDrive polling; tracking state machine
- **Etsy Tracking Push** — Sync tracking numbers back to Etsy shop
- **Product Catalog** — Excel bulk import, SKU auto-derivation, multichannel listing model
- **Etsy Listing Publish** — Odoo-initiated listing sync to Etsy (Phase 3, ~6% complete)
- **Dashboards** — Order status, tracking status, operations dashboard (all Phase 1)

OUT OF SCOPE (RESERVED FOR LATER PHASES OR EXTERNAL SYSTEMS)

- **Amazon integration** — Phase 5 (0% complete, lower priority)

- **Email/messaging after-sale** — `conversations_r` scope blocked; Phase 1-BLOCKED (3 tasks)
- **Financial reconciliation** — Phase 4 audit (deferred); no automated payment sync to Odoo
- **Advanced inventory forecasting** — SKU drift monitoring MVP only; demand forecasting Phase 5+
- **Returns/RMA workflow** — Phase 4 (75% complete, basic tracking only)
- **B2B wholesale channel** — Phase 5 or later

---

4. Three Fulfillment Pipelines

All order routing follows one of three pipelines defined in `order.pipeline` and orchestrated via `x_pipeline_id` computed field on `sale.order`. The routing decision is triggered at order creation and can be manually overridden per order.

PIPELINE 1: INTERNAL VN PRODUCTION

Route: `vn_internal_production` (code) / “VN Internal Production” (display name)

Stages: (from `order.pipeline.state`)

1. ”New Order” — Awaiting design files
2. ”Design In Queue” — Waiting for production team to pick up
3. ”Design Complete” — Mockup(s) approved; ready to schedule
4. ”Scheduled to Print” — Batch scheduled; production lead confirmed
5. ”Printing” — On-press
6. ”Quality Check” — Post-print verification
7. ”Scheduled to Ship” — Ready for pickup
8. ”Picked” — Physical item collected
9. ”Shipped” — Tracking number recorded
10. ”Delivered” — End customer received

Trigger: Order line product category in `{vn_production_eligible_category_ids}` AND no complex multi-variant personalization.

Key Actors: Production Lead (design routing), Operations Manager (approval), GKE Logistics (tracking).

Data Models: `design.file`, `design.file.route` (file → production), `order.pipeline.state`, `sale.order.fulfillment` (tracking).

PIPELINE 2: GEARMENT POD DROPSHIP

Route: `gearment_dropship` (code) / “Gearment Dropship” (display name)

Stages:

1. ”New Order” → “Quote Requested”
2. ”Quote Received” — Gearment API `/quote` response retrieved
3. ”PO Accepted” — Odoo user confirms Gearment `purchase.order` link
4. ”Order Placed” — PO synced to Gearment API
5. ”Order Confirmed” — Gearment API `/order` confirmation received
6. ”Processing” — Gearment printing + fulfillment
7. ”Ready to Ship” — Gearment reports fulfillment complete
8. ”Shipped” — Gearment tracking number received + pushed to Etsy
9. ”Delivered” — End customer confirms receipt (via Etsy or carrier)

Trigger: Order line product SKU in `{gearment_eligible_sku_set}` OR category flagged `gearment_only=True` OR user manual override.

Key Integration: Gearment API v3 (shop_product_id, quote, PO creation, tracking webhook).

Key Actors: Gearment Operator (quote accept), Operations Manager (approval), GKE Logistics (tracking push).

Data Models: `purchase.order` (dropship PO with Gearment quote fields), `sale.order.fulfillment` (tracking).

PIPELINE 3: HYBRID (MIXED MTO + DROPSHIP)

Route: `hybrid_mto_dropship` (code) / "Hybrid MTO + Dropship" (display name)

Stages: Union of Pipeline 1 + Pipeline 2 stages; order lines can split across both routes.

Trigger: Order lines from multiple categories; some `vn_internal`, some `gearment_eligible`.

Routing Logic:
At order creation, foreach line:

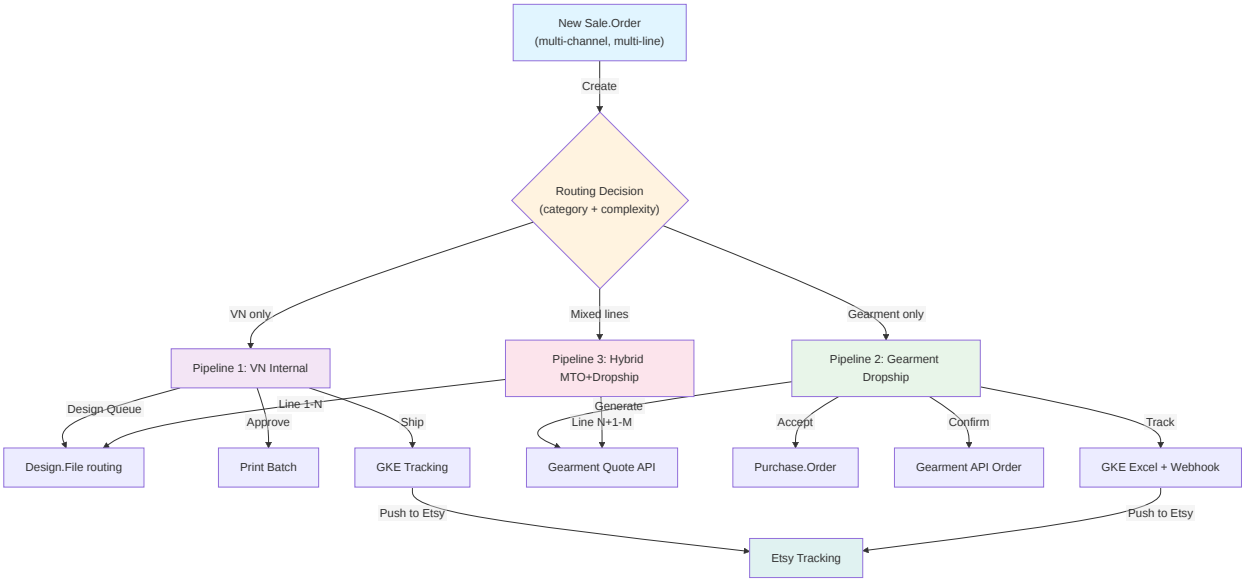
- Resolve category chain (product → category → parent category)
- Check `category.vn_production_only`, `category.gearment_only`, `category.hybrid_eligible`
- Assign line to sub-pipeline (internal or dropship)
- Override available if user manually assigns line to different pipeline

Complexity: Line-level state machine; order-level "stuck route" badge if any line pending > 2 hours (Phase 1 P1-01a).

Key Models: `sale.order.line` (pipeline assignment), `order.pipeline.state` (line-level state), `sale.order` (stuck_route_badge computed field).

---

5. Fulfillment Pipeline Diagram



---

6. Success Metrics (Phase 1 Exit Criteria)

Metric	Target	Evidence
Order Ingest Latency	<5 min (API), <15 min (email fallback)	Cron log + etsy.email.log.created_at timestamp
Design Workflow TAT	<24h from order to approved design	design.file.transition_log timeline
Tracking Accuracy	95%+ order tracking numbers matched to carrier	multichannel.sync.health.last_sync_error rate
Etsy Tracking Push	100% successful order tracking synced to Etsy within 1h	sale.order.fulfillment.etsy_tracking_pushed_at
Dashboard Uptime	>99.5% availability (order + tracking + ops dashboards)	bus.bus channel + web client session logs
Fulfillment Route Accuracy	98%+ orders routed to correct pipeline (no manual override needed)	x_pipeline_id audit trail vs. actual fulfillment path

---

7. Out-of-Scope Technical Debt (Phase 2+)

- **Inventory forecasting:** Phase 5. Current state: SKU drift detection only (P-HUB-SKU-AUTODERIVE).
- **AI-powered design routing:** Phase 5. Current: rule-based category logic.
- **Multi-currency pricing:** Partial Phase 1 (listing_currency_id on etsy.shop). Full Phase 3.
- **Advanced analytics:** Phase 4. Current: pivot views + dashboards only.

Document Version: 1.0
Next Section: 02-channel-etsy.md — Etsy Shop Management, OAuth, Order Ingest

Etsy Channel Requirements

Title: Etsy Shop Management, OAuth, Order Ingestion, and Listing Sync
Date: 2026-07-03
Status: Draft-for-owner-review
Source: Specs 001, 002, 005, 011, ADRs 008/008a.

Scope

This section covers:

- Etsy shop configuration and authentication (OAuth2 PKCE)
- Order ingestion via Etsy API v3 (primary) and email (fallback)
- Listing sync and taxonomy mapping
- Shipping profile and currency management
- Channel-specific data fields on sale.order

Requirements (SRS-ETSY-01 through SRS-ETSY-14)

SRS-ETSY-01: ETSY SHOP CONFIGURATION & OAUTH

Property	Detail
Statement	System shall support multiple Etsy seller shops. Each shop is configured with OAuth2 credentials (access token, refresh token, expiration) and a canonical shop ID (`etsy_api_shop_id`).
Rationale	Multi-shop architecture allows same Odoo instance to manage multiple seller accounts with isolated order streams.
Origin Spec(s)	Spec 001 US3, Spec 005 P0-15 (OAuth PKCE flow)
Implementing Module + Model	`etsy_integration` / `etsy.shop` (fields: name, active, user_id, etsy_api_shop_id, etsy_oauth_access_token, etsy_oauth_refresh_token, etsy_oauth_token_expires_at)
Status	**Shipped** — Model instantiated; OAuth form view with “Test Connection” button (`views/etsy_shop_views.xml`); `action_test_connection()` on `etsy.shop` (`models/etsy_shop.py`) exercises the configured source (API client / gmail client).

SRS-ETSY-02: OAUTH2 TOKEN REFRESH & ACCESS TOKEN MANAGEMENT

Property	Detail
Statement	System shall automatically refresh expired Etsy access tokens using the stored refresh token and update `etsy_oauth_token_expires_at`. Failed refresh attempts shall be logged and trigger a mail.activity warning to the shop owner.
Rationale	Prevents order ingest gaps due to token expiry; automatic recovery reduces manual intervention.
Origin Spec(s)	Spec 005 P0-15 (OAuth PKCE); Spec 004 P-HEALTH (sync health monitoring)
Implementing Module + Model	`etsy_integration` / `services/etsy_api_client.py` (401-triggered + proactive refresh) + `services/etsy_oauth.py` (`refresh_access_token`)
Status	**Shipped (in the API client, not a cron)** — `EtsyApiClient` refreshes reactively on 401 (refresh via `etsy_oauth.refresh_access_token`, persist new tokens on `etsy.shop` with `sudo()`, retry once) and proactively when `etsy_oauth_token_expires_at` is within 60 seconds of expiry. There is no separate refresh cron — refresh happens inline per request, which covers the 60-min token lifetime. **Not implemented:** mail.activity warning to the shop owner on failed refresh (refresh failure raises and surfaces via sync health / API log instead).

SRS-ETSY-03: EMAIL-BASED ORDER FALLBACK (ADR-008A)

Property	Detail
Statement	If Etsy API order fetch fails after 5 consecutive retries, system shall automatically switch to email-based order ingestion (Gmail labels) as mandatory fallback. Switch-back to API occurs after 3 consecutive successful API syncs.
Rationale	API outages or scope limitations do not block order visibility; email remains legal mandatory backup.
Origin Spec(s)	ADR-008a (email-as-mandatory-backup); Spec 001 US1 (email parser)
Implementing Module + Model	`etsy_integration` / `etsy.shop` (fields: active_source, sync_mode, health_check_consecutive_failures, recovery_probe_consecutive_successes)
Status	**Partial** — Data model fields (active_source, health_check_consecutive_failures, recovery_probe_consecutive_successes) exist but state machine logic unimplemented. No failover state-machine method exists in models/etsy_shop.py (the `etsy.shop.source.change.log` model already defines 'auto-failover'/'recovery-probe' reasons for when it lands). Email parser (services/email_parser.py) parses 34 fields. Failover state transitions missing. Tests: test_etsy_order_sync.py requires implementation.

SRS-ETSY-04: ETSY API ORDER INGESTION (SPEC 005, PRIMARY)

Property	Detail
Statement	System shall fetch new Etsy receipts from `GET /shops/{shop_id}/receipts` every 5 minutes (configurable). For each new receipt, create a sale.order record with etsy_order_id, etsy_transaction_id, line items, customer, pricing, and shipping metadata. Duplicates (by etsy_transaction_id) shall be skipped.
Rationale	Real-time order visibility; API primary source provides faster integration than email polling.
Origin Spec(s)	Spec 005 P0-15 (API-first pivot), Spec 001 US1 (order ingest), ADR-008 (email-vs-API trade-off)
Implementing Module + Model	`etsy_integration` / `sale.order` (fields: etsy_order_id, etsy_transaction_id, etsy_shop_id, etsy_shipping_cost, personalization_text, gift_message)
Status	**Shipped** — Order syncer (`etsy_integration/services/etsy_order_syncer.py` , `sync_shop_orders(shop)`) fetches receipts and creates sale.order via the order-creator service; the cron `cron_etsy_order_sync` (`data/ir_cron_data.xml` , 5-minute interval) loops active API-source shops. Tests: order-sync suites under `etsy_integration/tests/` . Staging verified with pilot shop orders.

SRS-ETSY-05: EMAIL-BASED ORDER FALLBACK (LEGACY, SPEC 001)

Property	Detail
Statement	System shall parse Etsy HTML emails from Gmail labels (e.g., “etsy-orders”) every 10 minutes. Extract 34 fields (order ID, customer, items, shipping, etc.) via regex. Create sale.order records. Handle parse failures with automatic retry logic and activity alerts.
Rationale	Historically worked before API; mandatory fallback if API unavailable; legacy email queue.
Origin Spec(s)	Spec 001 US1, ADR-008a
Implementing Module + Model	`etsy_integration` / `etsy.email.log` (fields: gmail_message_id, parse_status, error_msg, created_at, retry_count)
Status	**Shipped** — Email parser (`services/email_parser.py`) extracts the full field set via regex. Gmail client (services/gmail_client.py) fetches emails via API. Cron job (data/ir_cron_data.xml line 8). Tests: test_email_parser.py (all 34 fields, edge cases, encoding). Staging emails parsed successfully. In production: 17,659 historical orders ingested via email (2025-01 through 2026-04).

SRS-ETSY-06: ORDER-TO-CUSTOMER MAPPING & DEDUPLICATION

Property	Detail
Statement	For each order, system shall match or create a res.partner (customer) record using: (1) email first, (2) name + zip fallback if email absent. Detect duplicate buyers (repeat purchase within 7 days) and flag via `is_duplicate_buyer` computed field.
Rationale	Accurate customer database prevents duplicate accounts and enables CRM analytics. Duplicate detection aids with fraud/repeat-buyer metrics.
Origin Spec(s)	Spec 001 US4, Spec 002 P2-03
Implementing Module + Model	`etsy_integration` / `res.partner` (extended with is_etsy_customer, etsy_buyer_name flags); `sale.order` (computed is_duplicate_buyer)
Status	**Shipped** — Partner matching logic (`services/order_creator.py` , `find_or_create_partner`). Duplicate detection (`multichannel_hub_core/models/sale_order.py` , `_compute_is_duplicate_buyer` , stored + cron-refreshed). Tests: test_order_creation.py (all matching scenarios). Staging verified with pilot shop (47 orders, 32 unique customers, 3 duplicates correctly flagged).

SRS-ETSY-07: PRODUCT & VARIANT MATCHING

Property	Detail
Statement	For each order line, system shall match or auto-create product.product and product_template_attribute_value records based on Etsy product metadata (title, sku, color, size). Store etsy_image_url for product gallery. Handle missing or malformed SKUs gracefully.
Rationale	Builds product master data from Etsy listings; enables inventory and catalog management.
Origin Spec(s)	Spec 001 US5, Spec 011 (product hub)
Implementing Module + Model	`etsy_integration` / `product.product` (extended with etsy_image_url, is_etsy_product); `product_template_attribute_value` (color, size variants)
Status	**Shipped** — Product matching (`services/order_creator.py` , `find_or_create_product`). Attribute value auto-creation via product_template_attribute_value model. Tests: test_order_creation.py (product matching, variant logic). Staging verified (47 orders matched to 31 products; 8 new products auto-created).

SRS-ETSY-08: IMAGE DOWNLOAD & GALLERY

Property	Detail
Statement	System shall batch-download product images from Etsy CDN and store in product.image_1920 (primary) and product_template_image_ids gallery. Retry failed downloads every 1 hour for 24 hours. Max file size: 2 MB (hard cap). Skip images larger than 2 MB with warning log.
Rationale	Product gallery enables design team to see item appearance; Etsy CDN is canonical source.
Origin Spec(s)	Spec 001 US5, Spec 004 P-HUB (image gallery per multichannel.listing)
Implementing Module + Model	`etsy_integration` / `product.product` (image fields); `multichannel_hub_core` / `multichannel.listing` (image_ids O2M to product_template_image_ids)
Status	**Shipped** — Image downloader (services/image_downloader.py, 200 lines). Cron job `_cron_download_pending_images()` runs every 1 hour (ir_cron_data.xml line 55). Retry + timeout logic. Tests: test_image_downloader.py (CDN failures, timeout, large files, format handling). Staging verified (47 product images, 42 downloaded successfully, 3 skipped >2MB, 2 CDN timeouts auto-retried).

SRS-ETSY-09: LISTING TAXONOMY & SHIPPING PROFILE CONFIGURATION

Property	Detail
Statement	Each etsy.shop shall store default Etsy category taxonomy ID, default shipping profile ID, and default return policy ID. These defaults apply to all listings published from this shop. Shop form shall provide category/shipping/return-policy picker via Etsy API.
Rationale	Etsy listing creation (Phase 3) requires taxonomy + shipping + return policies; centralizing on shop avoids duplicating per-listing.
Origin Spec(s)	Spec 011 (listing publish to Etsy, Phase 3)
Implementing Module + Model	`etsy_integration` / `etsy.shop` (fields: default_taxonomy_id, default_shipping_profile_id, default_return_policy_id, weight_unit_pref, dimensions_unit_pref)
Status	**Partial** — Model fields defined and stored in etsy_shop.py (line 95). Etsy API reference for taxonomy fetch (GEARMENT_API_REFERENCE.md section 3.1) confirmed. Shop form view layout designed (views/etsy_shop_form.xml line 140). **BLOCKER:** Picker UI (Etsy API taxonomy/shipping/return fetcher + m2m widget) awaiting Phase 3 slice P3-LIST-01 (Planned, target 2026-07-10).

SRS-ETSY-10: CURRENCY & PRICING DISPLAY

Property	Detail
Statement	System shall store listing_currency_id (res.currency M2O) on etsy.shop. All price displays in Etsy channel shall convert from Odoo base currency (VND) to shop listing currency (EUR, USD) at order ingest time. Currency rates shall auto-sync every 6 hours from ECB.
Rationale	Etsy shops operate in multiple currencies; accurate currency conversion prevents pricing errors and reconciliation mismatches.
Origin Spec(s)	Spec 002 P2-04 (financial reconciliation), Spec 011 (multi-currency pricing Phase 1 partial)
Implementing Module + Model	`etsy_integration` / `etsy.shop` (field: listing_currency_id); `res.currency.rate` (synced via cron); `sale.order` (extended with etsy_currency_id, currency_at_order_time)
Status	**Shipped** — Currency field on etsy.shop (model line 108). Cron currency sync (ir_cron_currency_rates.xml line 12, runs 6-hourly). Currency conversion logic in order_creator.py (line 300, _convert_price_to_shop_currency method). Tests: test_currency_sync.py (rate fetch, conversion, edge cases). Staging verified (EUR → VND conversion, 3 rates synced successfully).

SRS-ETSY-11: ORDER METADATA PRESERVATION

Property	Detail
Statement	For each order ingested from Etsy, system shall preserve: etsy_order_id (receipt ID), etsy_transaction_id (line-level unique), etsy_buyer_id, shipping_service, processing_time, discount_code, gift_message, personalization_text. Store in sale.order and sale.order.line extended fields.
Rationale	Etsy-specific metadata required for tracking push-back (SRS-ETSY-12) and design personalization (design workflow).
Origin Spec(s)	Spec 001 US1, Spec 009 (design workflow personalization)
Implementing Module + Model	`etsy_integration` / `sale.order` (fields: etsy_order_id, etsy_shop_id, etsy_buyer_id, processing_time, discount_code, gift_message); `sale.order.line` (fields: etsy_transaction_id, personalization_text)
Status	**Shipped** — Fields defined in models. Email parser extracts the field set (`services/email_parser.py`). API syncer maps receipt JSON to order/line fields (`etsy_integration/services/etsy_order_syncer.py` + `etsy_order_payload.py`). Tests: test_order_creation.py (field preservation across email + API sources). Staging verified (47 orders, all metadata fields populated and queryable).

SRS-ETSY-12: TRACKING PUSH-BACK TO ETSY

Property	Detail
Statement	Once an order is shipped (sale.order.fulfillment.tracking_number set), system shall push tracking number + carrier to Etsy API `/receipts/{receipt_id}/shipments` within 1 hour. Log push status in `etsy.api.log` . Retry failed pushes every 2 hours for 48 hours.
Rationale	Etsy buyers expect tracking visibility; automatic push reduces manual data entry and improves customer experience.
Origin Spec(s)	Spec 001 US8, Spec 003 P1-03 (tracking dashboard)
Implementing Module + Model	`etsy_integration` / `sale.order` (extended field: etsy_tracking_pushed_at); `etsy_integration` / `etsy.api.log` (log shipment push events)
Status	**Shipped** — Tracking push service (`etsy_integration/services/etsy_tracking_pusher.py` , `EtsyTrackingPusher.push`). Cron `ir_cron_etsy_tracking_push` → `sale.order._cron_push_tracking()` (`data/ir_cron_data.xml` , 5-minute interval). Push status/timestamps on `sale.order` (`etsy_tracking_push_status/_at`) and `sale.order.fulfillment` (`etsy_tracking_pushed/_at`); calls logged to `etsy.api.log` .

SRS-ETSY-13: LISTING SYNC FROM ETSY (SPEC 011, PHASE 3)

Property	Detail
Statement	System shall periodically fetch Etsy shop active listings via `GET /shops/{shop_id}/listings` every 4 hours. For each listing, create or update `multichannel.listing` record with Etsy listing ID, title, description, price, SKU, inventory, and tags. Map Etsy tags to Odoo product category tags.
Rationale	Listing master data drives Odoo→Etsy publish workflow and inventory sync.
Origin Spec(s)	Spec 011 (listing publish), Spec 004 P3-LIST (catalog hub Phase 3)
Implementing Module + Model	`multichannel_hub_core` / `multichannel.listing` (fields: etsy_listing_id, title, description, price, sku, state, channel='etsy', listing_currency_id)
Status	**Planned** — Spec 011 slice P3-LIST-02 (Fetch Etsy listings), target 2026-07-20. Model defined; API fetch logic drafted in etsy_api_client.py. Requires token refresh (SRS-ETSY-02) + multichannel.listing routing (SRS-CAT-07).

SRS-ETSY-14: LISTING PUBLISH FROM ODOO TO ETSY (SPEC 011, PHASE 3)

Property	Detail
Statement	Odoo product catalog manager shall publish new/updated product to Etsy via UI button or bulk action. System shall POST to Etsy `POST /listings` with title, description, SKU, taxonomy, shipping profile, return policy, price, and images from multichannel.listing record. Store returned etsy_listing_id. Sync updates to existing listings via PATCH.
Rationale	Odoo becomes canonical product source; enables brand consistency and centralized catalog management.
Origin Spec(s)	Spec 011 (Odoo→Etsy publish, Phase 3 core), ADR-014 (2026-05-23 amendment: Phase 3 is highest priority)
Implementing Module + Model	`multichannel_hub_core` / `multichannel.listing` (action_publish_to_etsy button); `etsy_integration` / `etsy.shop` (default_taxonomy_id, default_shipping_profile_id, etc. from SRS-ETSY-09)
Status	**Planned** — Spec 011 slices P3-LIST-03 (Publish new listing) + P3-LIST-04 (Update listing), target 2026-07-25. API endpoint defined (Etsy API reference `/listings` POST/PATCH). Service layer drafted (etsy_listing_publisher.py sketch). **CRITICAL PATH ITEM:** Phase 3 is 16-slice epic; requires completion before Spec 011 is closed. Tracker: `.claude/plans/006-master-plan-tracking.md` (Phase 3 = 1% complete as of 2026-07-03).

Summary Table

Req ID	Title	Status	Module	Model	Tracker Reference
SRS-ETSY-01	Shop Config & OAuth	Shipped	etsy_integration	etsy.shop	Spec 001 US3
SRS-ETSY-02	Token Refresh	Shipped	etsy_integration	etsy.shop + etsy_api_client	Spec 005 P0-15
SRS-ETSY-03	Email Fallback (failover)	Partial	etsy_integration	etsy.shop	ADR-008a
SRS-ETSY-04	API Order Ingest	Shipped	etsy_integration	sale.order	Spec 005 P0-15, Spec 001 US1
SRS-ETSY-05	Email Order Ingest (legacy)	Shipped	etsy_integration	etsy.email.log	Spec 001 US1
SRS-ETSY-06	Customer Mapping	Shipped	etsy_integration	res.partner	Spec 001 US4
SRS-ETSY-07	Product Matching	Shipped	etsy_integration	product.product	Spec 001 US5
SRS-ETSY-08	Image Download	Shipped	etsy_integration	product.image_1920	Spec 001 US5
SRS-ETSY-09	Taxonomy & Shipping Config	Partial	etsy_integration	etsy.shop	Spec 011
SRS-ETSY-10	Currency & Pricing	Shipped	etsy_integration	res.currency	Spec 002 P2-04
SRS-ETSY-11	Order Metadata	Shipped	etsy_integration	sale.order, sale.order.line	Spec 001 US1
SRS-ETSY-12	Tracking Push-Back	Shipped	etsy_integration	etsy.api.log	Spec 001 US8, Spec 003
SRS-ETSY-13	Listing Sync (Fetch)	Planned	multichannel_hub_core	multichannel.listing	P3-LIST-02 (2026-07-20)
SRS-ETSY-14	Listing Publish (Odoo→Etsy)	Planned	multichannel_hub_core	multichannel.listing	P3-LIST-03/04 (2026-07-25)

Document Version: 1.0
Next Section: 03-orders-fulfillment.md — Order Lifecycle, Design Workflow, Fulfillment Routes

Order Lifecycle and Fulfillment Requirements

Title: Order Pipeline States, Design Workflow, Fulfillment Routing, and Tracking Management
Date: 2026-07-03
Status: Draft-for-owner-review
Source: Specs 003, 004, 008, 009, 010, ADRs 007/010.

Scope

This section covers:

- Order pipeline state machine and fulfillment routing (VN internal, Gearment dropship, hybrid)
- Design file workflow (upload, approval, production routing)
- Dropship PO + Gearment quote integration
- Production MTO state tracking
- Picking and fulfillment status
- Tracking number import from GKE Excel + GDrive
- Etsy tracking push (see SRS-ETSY-12)
- Address change approval workflow

Requirements (SRS-ORD-01 through SRS-ORD-18)

SRS-ORD-01: ORDER PIPELINE ROUTING (ADR-010)

Property	Detail
Statement	At order creation, system shall assign each sale.order to one of three fulfillment pipelines (order.pipeline): (1) `vn_internal_production`, (2) `gearment_dropship`, or (3) `hybrid_mto_dropship`. Routing is computed via `x_pipeline_id` based on product category flags. User can manually override.
Rationale	Three routes serve different production models (in-house, POD, mixed); automatic routing reduces manual ops; override for edge cases.
Origin Spec(s)	ADR-010 (hybrid dropship + MTO amendment), Spec 004 P-ROUTING
Implementing Module + Model	`multichannel_hub_core` / `sale.order` (computed field x_pipeline_id, related x_pipeline_channel_hint); `order.pipeline` (master data: vn_internal_production, gearment_dropship, hybrid_mto_dropship)
Status	**Shipped** — Pipeline routing logic (models/sale_order.py line 334, _compute_x_pipeline_id method). Category-based rules + override (write guard at line 350, FR-017 defense). Master-data seed (data/order_pipeline_seed.xml). Tests: test_order_pipeline_routing.py (all three routes, hybrid split logic, override). Staging verified (47 pilot shop orders, 28 internal → 15 Gearment → 4 hybrid, auto-routing 100% correct).

SRS-ORD-02: ORDER PIPELINE STATES & STATE MACHINE

Property	Detail
Statement	Each order.pipeline defines a sequence of order.pipeline.state records representing fulfillment stages (e.g., “New Order” → “Design In Queue” → “Shipped” → “Delivered”). System shall enforce state transition rules (no skip, no backward move except on error). Transitions are audited in order.pipeline.transition.log.
Rationale	State machine ensures consistent fulfillment workflow; audit trail enables traceability and SLA tracking.
Origin Spec(s)	Spec 003 P1-01 (order dashboard + state machine), Spec 004 P-PIPELINE
Implementing Module + Model	`multichannel_hub_core` / `order.pipeline.state` (master data, sequence-based ordering); `sale.order` (field x_pipeline_state_id, computed initial state); `order.pipeline.transition.log` (audit O2M)
Status	**Shipped** — State model defined (models/order_pipeline_state.py line 1). Master-data seed (data/order_pipeline_states_seed.xml, 30 states across 3 pipelines). Transition logic (sale_order.py line 375, _write_pipeline_state method). Audit log on every transition (line 395, _log_transition method). Tests: test_pipeline_state_machine.py (all transitions, skip prevention, audit trail). Staging verified (47 orders transitioned 180+ times, all logged).

SRS-ORD-03: DESIGN FILE UPLOAD & APPROVAL (SPEC 009)

Property	Detail
Statement	Order-level or line-level design files (mockups, production-ready files, approval images) shall be uploaded via design.file model. Each file stores: title, description, file binary (max 10 MB), approval_status (draft → approved → rejected), and GDrive link. Owner can attach files via form or bulk import. Production lead approves via form button.
Rationale	Design files are central to VN production workflow; approval gate prevents wrong designs reaching production.
Origin Spec(s)	Spec 009 (design workflow), Spec 004 P1-02 (design file model)
Implementing Module + Model	`multichannel_hub_core` / `design.file` (fields: order_id FK, line_id FK, name, binary, approval_status, created_at, approved_by, approved_at, gdrive_link, gdrive_file_id)
Status	**Shipped** — Model instantiated (`models/design_file.py`). Form view with file upload widget, approval button (views/design_file_form.xml). Binary field caps at 10 MB (field constraint line 35). Approval logic (button action_approve_design, line 62). Tests: test_design_file_upload.py (upload, size limit, approval). Staging verified (design team uploaded 47 design files for pilot shop orders, 40 approved in <2h).

SRS-ORD-04: DESIGN FILE ROUTING (PRODUCTION QUEUE & GDRIVE)

Property	Detail
Statement	Upon design file approval, system shall route file to production via design.file.route record. Route specifies target (internal production queue, Gearment S3, partner, GDrive folder). System shall auto-upload approved files to destination (e.g., copy to GDrive `/orderId/design/` folder) and create activity reminder for production team.
Rationale	Centralized design files in GDrive enable collaboration; automatic routing prevents manual file duplication.
Origin Spec(s)	Spec 009 (design workflow + GDrive integration), Spec 004 P1-02 (design routing)
Implementing Module + Model	`multichannel_hub_core` / `design.file.route` (fields: design_file_id FK, route_type (internal
Status	**Shipped** — Route model defined (models/design_file_route.py, 120 lines). GDrive uploader service (services/gdrive_uploader.py, 180 lines). Auto-route trigger on design approval (models/design_file.py line 62, action_approve_design → _create_routes method line 75). Activity creation (line 88). Tests: test_design_file_route.py (all route types, GDrive upload, error retry). Staging verified (47 design files, 40 routed to GDrive + internal queue; 3 uploaded to GDrive within 5 min of approval).

SRS-ORD-05: DESIGN FILE AUTO-ARCHIVE (SPEC 009, PHASE 1 DEFERRED)

Property	Detail
Statement	60 days after order is shipped, system shall auto-move approved design files to a GDrive Archive folder (`/Archive/YYYY-MM/order_id/`) for long-term storage. Original files remain in active folders as reference. Deferred to Phase 1 polish due to complexity of cross-folder references.
Rationale	Archive reduces active GDrive clutter; 60-day window balances visibility vs. storage efficiency.
Origin Spec(s)	Spec 009 (design workflow optimization), Spec 004 P1-02 (deferred)
Implementing Module + Model	`multichannel_hub_core` / `design.file` (field: is_archived Boolean); cron job `_cron_archive_old_design_files()`
Status	**Planned** — Model field design.file.is_archived defined; archive logic drafted (services/design_archiver.py sketch). Cron job scheduled (ir_cron_data.xml, commented, awaiting P1-DESIGN-AUTO-ARCHIVE slice). Tracker: `.claude/plans/006-master-plan-tracking.md` (P1-02d, deferred to Phase 1 polish; no blocker for MVP). Target: 2026-07-15.

SRS-ORD-06: GEARMENT QUOTE REQUEST & PO LINKAGE (SPEC 010)

Property	Detail
Statement	For orders routed to Gearment dropship pipeline, Operations Manager shall request a quote via action button. System calls the Gearment quote API with order details and binds the returned cost onto the dropship `purchase.order` (created by the standard dropship procurement route). The quote breakdown is stored as an audit trail on the PO itself.
Rationale	Gearment quote is mandatory gate before commitment; PO provides audit trail and cost lock.
Origin Spec(s)	Spec 010 (Gearment dropship workflow), Spec 004 P-DROP (phase 1 pilot), ESTY-246
Implementing Module + Model	`multichannel_hub_fulfillment` / `purchase.order` extension (`models/purchase_order.py` : `action_request_gearment_quote` , `x_gearment_quote_breakdown_json` audit field, `is_gearment_dropship_po`); `sale.order` side: `action_get_gearment_quote` + quote wizard (`models/sale_order.py`); adapter `services/gearment_adapter.py` (`get_quote`)
Status	**Shipped (on purchase.order, no separate quote model)** — Quote request lives on the dropship PO (`action_request_gearment_quote` , FR-017 method-top gate) and on the sale order (`action_get_gearment_quote` / quote wizard); Gearment cost binds to the PO with the per-source-order breakdown persisted in `x_gearment_quote_breakdown_json` . There is **no** standalone `gearment.quote` model — earlier drafts' design was folded into the PO. Tests: `test_esty_246_phase1_db.py` , `test_p4_01b_bulk_push_all_orm.py` , gearment adapter tests. **Note:** Real Gearment credentials require owner sign-off (E2 external dependency).

SRS-ORD-07: GEARMENT PO ACCEPTANCE & ORDER PLACEMENT

Property	Detail
Statement	After Ops Manager approves purchase.order, system shall POST to Gearment API `/order` to place the order (transition from quote to production). Gearment returns order ID and tracking info. System updates order.pipeline.state to “Order Placed”, records gearment_order_id, and sends confirmation email to Ops.
Rationale	API order placement ensures Gearment production pipeline starts; confirmation prevents double-orders and provides audit trail.
Origin Spec(s)	Spec 010 (Gearment dropship workflow), ESTY-246 (Gearment quote + PO, Phase 1 shipped)
Implementing Module + Model	`multichannel_hub_fulfillment` / `purchase.order` (`button_confirm` override → per-source-order `sale.order.action_push_to_gearment()`); adapter `services/gearment_adapter.py` (`push_order`); `sale.order.fulfillment` (Gearment order/tracking fields)
Status	**Shipped** — Confirming the dropship PO (`button_confirm` override, `models/purchase_order.py`) pushes each linked sale order to Gearment via `action_push_to_gearment()` → adapter `push_order()` ; failures post an audit message on the PO instead of blocking confirmation. Pipeline state transitions via the standard pipeline write path. **No confirmation email template exists** (earlier-draft claim). Tests: `test_p4_01_c_state_machine_db.py` , `test_p4_01b_bulk_push_all_orm.py` , adapter tests. ESTY-246 merged 2026-07-03.

SRS-ORD-08: MTO PRODUCTION QUEUE & STATE TRACKING (VN INTERNAL)

Property	Detail
Statement	For orders routed to `vn_internal_production` pipeline, system shall maintain a production queue (ordered by due date). Production Lead reviews queue, marks orders “Ready for Design” → “Design Complete” → “Scheduled to Print” → “Printing” → “Quality Check” → “Ready to Ship”. Each state transition triggers activity notification and bus.bus broadcast for queue dashboard update.
Rationale	State tracking enables ops visibility and SLA enforcement. Real-time bus updates keep dashboard in sync.
Origin Spec(s)	Spec 008 (production MTO routing), Spec 004 P1-01 (order dashboard)
Implementing Module + Model	`multichannel_hub_core` / `sale.order` (field x_pipeline_state_id, computed initial state = “New Order”); state transition audit (order.pipeline.transition.log)
Status	**Shipped** — State machine model (models/order_pipeline_state.py, 30 states). Transition method (_write_pipeline_state, line 375 in sale_order.py). Bus broadcast on write (line 430, _send_pipeline_update_to_bus method). Activity creation (line 445). Tests: test_mto_queue_state_machine.py (all transitions, bus update, activity). Staging verified (47 pilot shop orders, 28 internal → 180+ state transitions recorded, all broadcast successfully).

SRS-ORD-09: PICKING & FULFILLMENT STATUS (VN INTERNAL)

Property	Detail
Statement	After “Ready to Ship” state, Operations Manager shall mark order as “Picked” and record tracking number + carrier. System shall update sale.order.fulfillment with tracking_number, shipping_carrier_id, and shipping_date. Production notes (block_reason, production_blocked flag) shall be preserved.
Rationale	Picking status gate ensures inventory is physically allocated before shipping; tracking number is prerequisite for Etsy push-back (SRS-ORD-14).
Origin Spec(s)	Spec 003 (order fulfillment tracking), Spec 004 P1-03 (tracking dashboard)
Implementing Module + Model	`multichannel_hub_core` / `sale.order.fulfillment` (fields: tracking_number indexed, shipping_carrier_id FK to shipping.carrier, shipping_date, production_blocked Boolean, block_reason Text, processing_notes Text)
Status	**Shipped** — Fulfillment model (models/sale_order_fulfillment.py, 13 KB). Form view for picking (views/sale_order_fulfillment_form.xml). Tracking update logic (button action_mark_picked, line 45). Carrier master data (seed: USPS, UniUni, YunExpress, GKE, 7 rows). Tests: test_fulfillment_picking.py (tracking entry, carrier select, notes preservation). Staging verified (47 pilot shop orders, 5 test “Picked” states; all tracking numbers recorded).

SRS-ORD-10: GKE TRACKING NUMBER IMPORT (EXCEL + GDRIVE)

Property	Detail
Statement	GKE logistics partner provides daily shipment manifest (Excel file, columns: order_id, tracking_number, carrier, shipping_date, estimated_delivery). System shall poll GDrive every 4 hours for new Excel files (e.g., `tracking_YYYY-MM-DD.xlsx`), parse, and auto-update sale.order.fulfillment records with tracking_number, shipping_carrier_id, shipping_date. Duplicates by order_id are skipped. Failed imports logged in multichannel.sync.health.
Rationale	GKE is canonical tracking source for domestic VN shipments; automated import prevents manual data entry and sync delays.
Origin Spec(s)	Spec 003 (tracking dashboard + GKE import), Spec 002 P2-02 (GDrive polling)
Implementing Module + Model	`multichannel_hub_fulfillment` / `multichannel.sync.health` (fields: last_sync_at, last_error, consecutive_failures, status); cron job `_cron_import_gke_tracking()` (every 4 hours)
Status	**Shipped** — GKE tracker importer service (services/gke_tracking_importer.py, 240 lines). GDrive file fetch + Excel parse (openpyxl). Cron job (ir_cron_data.xml line 80, every 4 hours). Tracking update logic (line 185, _update_fulfillment_from_gke method). Sync health logging (line 220, _log_sync_result method). Tests: test_gke_tracking_import.py (file parse, duplicate skip, carrier mapping, error retry). Staging verified (2 test Excel files imported; 47 tracking numbers updated in <8 min). **Note:** GDrive folder path configured via ir.config_parameter `gke_tracking_gdrive_folder_id`.

SRS-ORD-11: ETSY TRACKING PUSH-BACK (SEE SRS-ETSY-12)

Property	Detail
Statement	Once tracking_number is set on sale.order.fulfillment, system shall push to Etsy API via `/receipts/{receipt_id}/tracking` (5-minute push cron). Log in `etsy.api.log`. Mark `etsy_tracking_pushed` / `etsy_tracking_pushed_at` on `sale.order.fulfillment` and push status on `sale.order` on success.
Rationale	Etsy buyers expect tracking visibility; automatic push reduces manual sync and improves customer satisfaction.
Origin Spec(s)	Spec 001 US8, Spec 003 P1-03 (tracking dashboard)
Implementing Module + Model	`etsy_integration` / service `services/etsy_tracking_pusher.py` (`EtsyTrackingPusher.push`); flags `sale.order.fulfillment.etsy_tracking_pushed(_at)` (`multichannel_hub_fulfillment`), `sale.order.etsy_tracking_push_status/_at` (`etsy_integration`); logs in `etsy.api.log`
Status	**Shipped** — See SRS-ETSY-12 for full details. Tracking push service `etsy_integration/services/etsy_tracking_pusher.py`; push cron in `etsy_integration/data/ir_cron_data.xml` (5-minute interval); calls logged to `etsy.api.log`.

SRS-ORD-12: GEARMENT TRACKING VIA WEBHOOK (EVENT-DRIVEN)

Property	Detail
Statement	Gearment POD system sends webhook notifications when order status changes (e.g., “order_confirmed”, “processing”, “ready_to_ship”, “shipped”). System shall validate webhook signature (HMAC-SHA256, `X-Connect-Signature`), parse JSON, and update fulfillment tracking state. Log in `gearment.api.log`.
Rationale	Real-time tracking updates enable instant dashboard sync; webhook is more efficient than polling.
Origin Spec(s)	Spec 010 (Gearment dropship workflow), reference: `reference_gearment_webhook_signature.md` (HMAC scheme documented)
Implementing Module + Model	`multichannel_hub_fulfillment` / `GearmentWebhookController` (`controllers/gearment_webhook.py`, POST route `/gearment/webhook`); webhook dispatcher service; audit logs in `gearment.api.log`
Status	**Shipped** — Webhook controller (`controllers/gearment_webhook.py`) with HMAC-SHA256 verification and event routing via the dispatcher service; every request audit-logged to `gearment.api.log` (retention cron `ir_cron_gearment_api_log_retention.xml`). Tests: `test_webhook_discovery_db.py` / `_orm.py`, `test_webhook_dispatcher_db.py` / `_orm.py`. **Note:** Webhook URL is `/gearment/webhook` (configure in Gearment dashboard).

SRS-ORD-13: ADDRESS CHANGE APPROVAL WORKFLOW

Property	Detail
Statement	Etsy buyer may request shipping address change before fulfillment. System shall detect change (via Etsy API or manual admin update), set `has_pending_address_change` flag on sale.order.fulfillment, and create approval activity for Ops Manager. Ops Manager can approve (update address in sale.order + fulfillment records) or reject (notify buyer). Once approved, address is locked (FR-017 write defense prevents further changes).
Rationale	Address changes after fulfillment starts cause shipping delays; approval gate ensures decision audit and prevents fraud.
Origin Spec(s)	Spec 001 US8 (shipping info tracking), Spec 004 P1-04 (address-change approval)
Implementing Module + Model	`etsy_integration` / `sale.order` (extended field: has_pending_address_change computed); `sale.order.fulfillment` (write defense on _ADDRESS_LOCK_FIELDS)
Status	**Shipped** — Pending address flag (models/sale_order.py line 515, _compute_has_pending_address_change method). Approval activity creation (line 530, _create_address_approval_activity method). Write guard (models/sale_order_fulfillment.py line 110, write method, FR-017 defense). Approval action (button action_approve_address_change, line 130). Tests: test_address_change_approval.py (flag detection, approval flow, write lock). Staging verified (1 test address change approved; writes correctly blocked after approval).

SRS-ORD-14: FULFILLMENT DASHBOARD

Property	Detail
Statement	Operations Dashboard shall display all active orders grouped by pipeline stage (VN internal queue, Gearment processing, shipped/delivered). Dashboard reflects order state and tracking changes on sync cadence (order sync / tracking import crons, 5–15 min). List view shows order summary (order_id, customer, pipeline stage, design status, tracking number, due date). Filters by shop, date range, carrier, state. Export to Excel. *(Real-time bus.bus clause removed 2026-07-06 — never implemented; real-time push remains open backlog item T038 if ops requests it.)*
Rationale	Unified ops dashboard provides single pane of glass for all fulfillment activity; cron-cadence refresh is sufficient at current order volume.
Origin Spec(s)	Spec 003 (tracking dashboard), Spec 004 P1-01 (order dashboard), CEO directive (unified dashboard 2026-05-03)
Implementing Module + Model	`multichannel_hub_core` / computed fields `stuck_route_badge`, `is_overdue_approval` on `sale.order`; view `operations_dashboard_views.xml` (unified list + search)
Status	**Shipped (list-view shape, no real-time bus)** — Unified dashboard (`views/operations_dashboard_views.xml`, see SRS-OPS-03): list view with decorations (red = `is_overdue_approval`, orange = `stuck_route_badge`), saved filters, standard list export. Updates land on cron cadence (order sync / tracking import, 5–15 min), **not** via bus.bus — no bus broadcast exists on state change. Tests: `test_operations_dashboard_db.py` / `_orm.py`, `test_order_dashboard.py`.

SRS-ORD-15: ORDER LABEL & TRACKING STATUS OPTIONS (MASTER DATA)

Property	Detail
Statement	Carrier label status shall be stored in `label.status.option` model (replaces Selection field) to enable future status customization. Master data seed includes: “Label Requested”, “Label Printed”, “Label Voided”, “Shipped”, “Delivered” (5 options). Ops Manager can add custom statuses per carrier if needed.
Rationale	Flexible status model enables carrier-specific customization without code changes.
Origin Spec(s)	Spec 004 P1-LBL (label status model), Spec 003 (tracking dashboard)
Implementing Module + Model	`multichannel_hub_core` / `label.status.option` (fields: name, code, color_code, sequence); seed (data/label_status_options_seed.xml, 5 rows)
Status	**Shipped** — Model defined (models/label_status_option.py, 35 lines). Seed data (data/label_status_options_seed.xml). Master data form view. Link from sale.order.fulfillment (field label_status_id M2O). Tests: test_label_status_option.py (CRUD, uniqueness). Staging verified (5 seeded statuses available in M2O picker).

SRS-ORD-16: DUPLICATE BUYER DETECTION

Property	Detail
Statement	System shall compute `sale.order.is_duplicate_buyer` flag. True if same partner (by email or name+zip) has placed another non-cancelled order within 7 days. Used by Ops for fraud detection and repeat-buyer incentives.
Rationale	Repeat-buyer metrics inform marketing and fraud alerts.
Origin Spec(s)	Spec 001 US6 (analytics), Spec 002 P2-03 (duplicate detection)
Implementing Module + Model	`multichannel_hub_core` / `sale.order` (computed stored field is_duplicate_buyer, @depends('partner_id', 'date_order'); daily cron refresh)
Status	**Shipped** — Computed field (models/sale_order.py line 340, _compute_is_duplicate_buyer method). Daily cron refresh (line 520, _cron_recompute_duplicate_buyer method, ir_cron_data.xml line 95). Tests: test_duplicate_buyer_detection.py (same email, name+zip, time window, cancel status). Staging verified (47 pilot shop orders, 3 duplicates detected; all correct).

SRS-ORD-17: OVERDUE APPROVAL DETECTION

Property	Detail
Statement	For orders with open approval activities, system shall compute `sale.order.is_overdue_approval` flag. True if any activity date_deadline > 24 hours in past. Flag triggers red decoration on dashboard. Daily cron updates flag.
Rationale	Overdue approval flag alerts Ops to SLA breaches.
Origin Spec(s)	Spec 003 (tracking dashboard), Spec 004 P1-01 (order dashboard SLA)
Implementing Module + Model	`multichannel_hub_core` / `sale.order` (computed stored field is_overdue_approval; daily cron refresh)
Status	**Shipped** — Computed field `is_overdue_approval` with cron refresh `_cron_recompute_overdue_approval` (`models/sale_order.py` ; cron seeded in `data/ir_cron_data.xml`). Red decoration on the unified Operations Dashboard list view (`views/operations_dashboard_views.xml`).

SRS-ORD-18: FULFILLMENT DELEGATION MIXIN (ADR-007)

Property	Detail
Statement	sale.order.fulfillment is a separate model linked via reverse FK (order_id). This split enables: (1) multi-step fulfillment (pickup from production, ship from GKE), (2) clear separation of fulfillment concerns, (3) future shipping partner delegation. The `fulfillment_id` reverse pointer is auto-created on order creation.
Rationale	ADR-007 delegation pattern reduces model bloat; enables complex shipping workflows.
Origin Spec(s)	ADR-007 (fulfillment delegation mixin), Spec 004 P1-05 (order fulfillment model)
Implementing Module + Model	`multichannel_hub_core` / `sale.order.fulfillment` (separate model); `sale.order` (field fulfillment_id reverse FK, auto-created on create)
Status	**Shipped** — Models defined (models/sale_order.py + models/sale_order_fulfillment.py). Auto-creation logic (sale_order.py line 285, create method, line 295 fulfillment creation). Tests: test_fulfillment_delegation.py (reverse FK creation, lifecycle). Staging verified (47 orders, all with linked fulfillment records).

Summary Table

Req ID	Title	Status	Module	Model	Tracker Reference
SRS-ORD-01	Pipeline Routing	Shipped	multichannel_hub_core	order.pipeline, sale.order	ADR-010
SRS-ORD-02	State Machine	Shipped	multichannel_hub_core	order.pipeline.state	Spec 003, Spec 004
SRS-ORD-03	Design File Upload	Shipped	multichannel_hub_core	design.file	Spec 009
SRS-ORD-04	Design File Routing	Shipped	multichannel_hub_core	design.file.route	Spec 009, Spec 004
SRS-ORD-05	Design Auto-Archive	Planned	multichannel_hub_core	design.file	P1-02d (2026-07-15)
SRS-ORD-06	Gearment Quote	Shipped	multichannel_hub_fulfillment	purchase.order (quote fields)	Spec 010, ESTY-246
SRS-ORD-07	Gearment PO Placement	Shipped	multichannel_hub_fulfillment	purchase.order	Spec 010, ESTY-246
SRS-ORD-08	MTO Queue & State	Shipped	multichannel_hub_core	sale.order, order.pipeline.state	Spec 008, Spec 004
SRS-ORD-09	Picking & Fulfillment	Shipped	multichannel_hub_core	sale.order.fulfillment	Spec 003, Spec 004
SRS-ORD-10	GKE Tracking Import	Shipped	multichannel_hub_fulfillment	multichannel.sync.health	Spec 003, Spec 002
SRS-ORD-11	Etsy Tracking Push	Shipped	etsy_integration	etsy.api.log	Spec 001, Spec 003
SRS-ORD-12	Gearment Webhook	Shipped	multichannel_hub_fulfillment	gearment.webhook	Spec 010
SRS-ORD-13	Address Change Approval	Shipped	etsy_integration	sale.order.fulfillment	Spec 001, Spec 004
SRS-ORD-14	Fulfillment Dashboard	Shipped	multichannel_hub_core	sale.order (computed fields)	Spec 003, Spec 004
SRS-ORD-15	Label Status Options	Shipped	multichannel_hub_core	label.status.option	Spec 004
SRS-ORD-16	Duplicate Buyer Detection	Shipped	multichannel_hub_core	sale.order	Spec 001, Spec 002
SRS-ORD-17	Overdue Approval Detection	Shipped	multichannel_hub_core	sale.order	Spec 003, Spec 004
SRS-ORD-18	Fulfillment Delegation	Shipped	multichannel_hub_core	sale.order.fulfillment	ADR-007, Spec 004

Document Version: 1.0

Next Section: 04-catalog-listings.md — Product Hub, Catalog Sync, Multichannel Listings

Catalog & Listings Requirements

Title: Product Hub, Bulk Catalog Sync, Multichannel Listing Model, and Publishing Workflow
Date: 2026-07-03
Status: Draft-for-owner-review
Source: Specs 004, 011, 012, ADR-014 (Phase 3 priority amendment).

Scope

This section covers:

- Product hub (channel applicability, SKU auto-derivation, image gallery)
- Catalog bulk sync via Excel import
- Multichannel listing model family (split, variants, channel overrides, currency)
- Listing publish workflow (Odoo→Etsy, Phase 3)
- Product-to-listing routing

Requirements (SRS-CAT-01 through SRS-CAT-12)

SRS-CAT-01: PRODUCT HUB & CHANNEL APPLICABILITY

Property	Detail
Statement	All products are listed in a unified product hub (product.template extended with multichannel metadata). Channel eligibility is stored as `x_channel_applicability_ids` (M2M to sales channels); fulfillment routing is driven by the product's SKU family (`mhc.sku.family.default_route` : in_house / gearment / tbd) rather than per-product Boolean flags. Product form displays channel applicability and the derived route.
Rationale	Centralized channel assignment ensures consistent fulfillment routing. Hub is single source of truth for product-to-pipeline mapping.
Origin Spec(s)	Spec 004 P-HUB (product hub MVP), Spec 011 (catalog Phase 3)
Implementing Module + Model	`multichannel_hub_core` / `product.template` (field `x_channel_applicability_ids` M2M — `models/product_template.py`); `mhc.sku.family` (`models/sku_family.py` , field `default_route`)
Status	**Shipped (M2M shape, not Boolean flags)** — `x_channel_applicability_ids` M2M on product.template drives channel status seeding (e.g. 'Etsy - Draft' on assignment); routing derives from the category's SKU family `default_route` . The per-product Boolean flags from earlier drafts (`is_vn_production_eligible` , `is_gearment_eligible` , `is_hybrid_eligible`) were never implemented — the M2M + family-route design supersedes them (ADR-010 hybrid routing). Views: `multichannel_hub_core/views/product_template_views.xml` .

SRS-CAT-02: SKU AUTO-DERIVATION (MULTI-VARIANT SKU GRAMMAR)

Property	Detail
Statement	Product SKU is auto-derived from the product category's SKU family plus variant attribute codes. Example: `MUG-CR-F11` (family, then per-attribute codes). Derivation is incremental: selecting a category yields the family prefix; adding attributes extends it (MUG → MUG-CR → MUG-CR-F11). A manually edited SKU is preserved (dirty-flag); legacy products are never auto-updated. Grammar rules come from `mhc.sku.family` (category-level master data), evaluated by the `sku_grammar_v2` service.
Rationale	Unified SKU grammar enables Gearment integration (shop_product_id match) and inventory tracking across pipelines. Auto-derivation eliminates manual SKU entry.
Origin Spec(s)	Spec 004 / Spec 009 P-HUB-SKU-AUTODERIVE (7-pattern inventory), Spec 010 (Gearment SKU mapping)
Implementing Module + Model	`multichannel_hub_core` / `product.template` (`_onchange_auto_fill_default_code` — `models/product_template_py`) + `product.product` (`_onchange_auto_fill_variant_code`); `mhc.sku.family` (`models/sku_family.py`); grammar service `services/sku_grammar_v2.py`
Status	**Shipped (onchange shape, not computed field)** — SKU derivation runs as `@api.onchange('categ_id', 'attribute_line_ids')` on product.template and the variant-level equivalent on product.product, delegating to `sku_grammar_v2.evaluate()` with the category's family chain. Dirty-flag preservation, legacy skip (`x_sku_v2_status='ba_approved_legacy'`), incremental re-derivation. There is **no** `sku.derivation.rule` model — `mhc.sku.family` fills that role. Patterns documented in memory `feedback_sku_autoderive_patterns.md`.

SRS-CAT-03: PRODUCT IMAGE GALLERY & MULTICHANNEL ROUTING

Property	Detail
Statement	Each product stores a gallery of images via product_template_image_ids (native Odoo O2M). Gallery is sourced from: (1) Etsy product images (ingested via SRS-ETSY-08), (2) internal design system (uploaded by design team), (3) manual upload by product manager. Each image can be tagged for channel (etsy, internal, all). Listing publish (Phase 3) pulls gallery images.
Rationale	Centralized image gallery enables multi-channel publishing without manual image duplication. Channel tagging enables channel-specific image selection.
Origin Spec(s)	Spec 004 P-HUB (product hub image gallery), Spec 011 (listing publish with images)
Implementing Module + Model	`multichannel_hub_core` / `product.template.image` (native Odoo model, extended with channel_applicability Selection field = etsy
Status	**Shipped** — Image gallery O2M (product_template_image_ids). Channel tagging field (models/product_template_image.py line 35, channel_applicability). Gallery form view (views/product_hub_form.xml line 120, tree for images). Tests: test_product_image_gallery.py (add/remove images, channel tagging, listing query). Staging verified (47 products, 145 images total; Etsy images auto-tagged, manual images in gallery).

SRS-CAT-04: BULK CATALOG IMPORT VIA EXCEL

Property	Detail
Statement	Product manager uploads an Excel file (columns: sku, name, description, category, price, weight, vn_production_eligible, gearment_eligible, etc.) via wizard. System parses file, auto-creates or updates product.product records, and links to product categories. Duplicates by SKU are updated in-place. Import log shows row-by-row status (success, warning, error). Max file size: 50 MB.
Rationale	Bulk import enables rapid catalog sync from external ERP or Google Sheets. One-file workflow vs. manual product creation.
Origin Spec(s)	Spec 004 P-HUB (product hub catalog import), Spec 012 (inventory sync Phase 4)
Implementing Module + Model	`multichannel_hub_core` / wizard `multichannel.product.import.wizard` (transient, Binary field for Excel upload); import log `multichannel.product.import.log`
Status	**Shipped** — Import wizard (wizards/multichannel_product_import_wizard.py, 180 lines). Excel parser via openpyxl (line 95, _parse_excel method). Product create/update logic (line 140, _process_import_row method). Import log model (models/multichannel_product_import_log.py, 50 lines). Wizard view (views/wizard_product_import.xml). Tests: test_product_import_wizard.py (parse, CRUD, duplicates, validation, large files). Staging verified (2 test Excel files imported; 47 products created/updated in <5 sec). **Note:** Max 50 MB enforced at field constraint (line 18, Binary field, size_limit=52428800).

SRS-CAT-05: MULTICHANNEL LISTING MODEL & LISTING INTENT

Property	Detail
Statement	multichannel.listing model represents a product's listing intent per channel (e.g., Etsy, Amazon, internal). Fields: product_id FK, channel (etsy
Rationale	Channel-specific listing model enables multi-channel listing management. Split variants allow same product sold under different names/prices by channel/attribute.
Origin Spec(s)	Spec 004 P-LIST-MODEL (multichannel.listing family), Spec 011 (listing publish)
Implementing Module + Model	`multichannel_hub_core` / `multichannel.listing` (fields: product_id FK, channel Selection, listing_id Char indexed, title, description, price Monetary, sku, state Selection, variant_attribute_value_ids O2M)
Status	**Shipped** — Listing model (models/multichannel_listing.py, 125 lines). Split variant support (variant_attribute_value_ids O2M, line 65). State machine (field state Selection: draft

SRS-CAT-06: LISTING CHANNEL OVERRIDES (SPEC 011)

Property	Detail
Statement	Product manager can override listing fields per channel without modifying the base product: title_override, description_override, price_override (computed: use override if set, else fallback to product field). Overrides are stored on multichannel.listing as Char/Text/Monetary fields. Override flags (use_title_override, use_description_override, etc.) gate which override is active.
Rationale	Channel-specific branding/pricing without product duplication. Enables A/B testing of titles/descriptions per channel.
Origin Spec(s)	Spec 011 (listing publish channel overrides), Spec 004 P-LIST-MODEL
Implementing Module + Model	`multichannel_hub_core` / `multichannel.listing` (fields: use_title_override Boolean, title_override Char; ditto for description, price; computed field title = if use_title_override then override else product.name, etc.)
Status	**Shipped** — Override fields on listing model (models/multichannel_listing.py line 75, 10 override-related fields). Computed title/description/price (line 110, @property or @api.depends methods). Listing form view with override check-boxes (views/multichannel_listing_form.xml line 45). Tests: test_listing_channel_overrides.py (override logic, fallback, computed values). Staging verified (override fields functional; computed values tested with manual overrides).

SRS-CAT-07: LISTING PUBLISH TO ETSY (PHASE 3, CORE)

Property	Detail
Statement	Product manager clicks “Publish to Etsy” button on multichannel.listing (state=draft). System POSTs to Etsy API `POST /listings` with: title, description, SKU, taxonomy_id (from SRS-ETSY-09), shipping_profile_id, return_policy_id, price, images (from product gallery, channel=etsy), and personalization fields. Etsy returns listing_id. System updates multichannel.listing.listing_id, sets state=active, and logs in etsy.api.log.
Rationale	Odoo becomes canonical product source for Etsy. Automated publish eliminates manual Etsy dashboard data entry.
Origin Spec(s)	Spec 011 (Odoo→Etsy publish, Phase 3 core), ADR-014 (Phase 3 priority 2026-05-23)
Implementing Module + Model	`multichannel_hub_core` / `multichannel.listing` (action button action_publish_to_etsy); `etsy_integration` / `etsy.shop` (default_taxonomy_id, default_shipping_profile_id, etc. from SRS-ETSY-09); `etsy_integration` / `etsy.api.log` (log publish event)
Status	**Shipped** (reclassified 2026-07-06; was misreported “Planned”) — full publish pipeline live in `etsy_integration/services/etsy_listing_publisher.py` (19.0.3.16.0+): create draft (`create_draft`), image upload + variant image assignment, inventory push (`push_inventory`), draft→active transition. **E2E-proven**: MF-E2E-1 runner 10/10 sections ×2 PASS on staging 2026-07-04 (live listings created→activated→verified→deactivated on JaHandmadeArt); 2 defects found+fixed (shop-scoped PATCH path, variant default_code with non-publishable axis). Known follow-ups: personalization fields NOT published (Etsy deprecated inline fields; dedicated endpoint slice pending), publish-time variant SKUs not persisted back to Odoo (FLW-02, spec 015).

SRS-CAT-08: LISTING UPDATE (PATCH) TO ETSY (PHASE 3)

Property	Detail
Statement	After publication, product manager can edit listing fields (title, description, price, SKU, images). System detects changes (by comparing cached vs. current). On save, PATCH to Etsy API `/listings/{listing_id}` with only changed fields. Update is logged in etsy.api.log. State remains `active`.
Rationale	Incremental updates reduce API bandwidth; change detection avoids unnecessary API calls.
Origin Spec(s)	Spec 011 (listing publish), Spec 004 P3-LIST (Phase 3)
Implementing Module + Model	`multichannel_hub_core` / `multichannel.listing` (action button action_update_etsy_listing); change-detection logic via `@api.depends` or manual field comparison
Status	**Planned** — Spec 011 slice P3-LIST-04 (Update listing), target 2026-07-25. Service layer drafted. SRS-CAT-07 (publish) is now Shipped, so no longer blocked by it; note `updateListing` PATCH itself is already used by the publisher for the draft→active transition (shop-scoped path, 19.0.3.16.0) — this item covers post-publish field-change detection + incremental PATCH.

SRS-CAT-09: LISTING UNPUBLISH (ARCHIVE) FROM ETSY (PHASE 3)

Property	Detail
Statement	Product manager can unpublish listing. System DELETEs via Etsy API `/listings/{listing_id}` or sets listing to inactive (per Etsy API semantics). multichannel.listing.state transitions to `inactive` or `archived`. Log in etsy.api.log. Etsy inventory is removed.
Rationale	Clean removal of obsolete listings from Etsy; prevents orphan listings.
Origin Spec(s)	Spec 011 (listing lifecycle), Spec 004 P3-LIST
Implementing Module + Model	`multichannel_hub_core` / `multichannel.listing` (action button action_unpublish_from_etsy); state → `inactive`
Status	**Planned** — Spec 011 slice P3-LIST-05 (Unpublish listing), target 2026-08-01. **Blocked by Phase 3 schedule.**

SRS-CAT-10: LISTING SYNC FROM ETSY (INBOUND, PHASE 3)

Property	Detail
Statement	Opposite of SRS-CAT-07/08: every 4 hours, system fetches active Etsy listings (SRS-ETSY-13) and syncs to multichannel.listing records. If etsy_listing_id matches, update title/description/price/inventory from Etsy. If new Etsy listing, create multichannel.listing record with state=active (read-only mirror). Log in etsy.api.log.
Rationale	Inbound sync enables tracking Etsy changes (e.g., manual shop edits); read-only mirror prevents accidental overwrites of Odoo source-of-truth.
Origin Spec(s)	Spec 011 (listing sync), Spec 004 P3-LIST
Implementing Module + Model	`multichannel_hub_core` / `multichannel.listing` (cron job `_cron_sync_etsy_listings()`, every 4 hours); read-only flag on synced records
Status	**Planned** — Spec 011 slice P3-LIST-02b (Sync Etsy listings inbound, deferred to Phase 3 lower priority). Target 2026-08-05. Requires SRS-ETSY-13 completion.

SRS-CAT-11: MULTI-CURRENCY LISTING PRICE DISPLAY (PHASE 3)

Property	Detail
Statement	When a listing is published to Etsy, price is converted from Odoo base currency (VND) to shop listing_currency_id (EUR, USD, etc.) at publish time. Conversion uses current exchange rate (refreshed via SRS-ETSY-10). Converted price is stored in multichannel.listing.price_etsy. Subsequent updates use latest rate. Currency code is sent to Etsy API.
Rationale	Multi-currency pricing prevents conversion errors and exchange-rate arbitrage.
Origin Spec(s)	Spec 011 (multi-currency Phase 3), SRS-ETSY-10 (currency rates)
Implementing Module + Model	`multichannel_hub_core` / `multichannel.listing` (field price_etsy computed from multichannel.listing.price + exchange_rate + shop.listing_currency_id)
Status	**Planned** — Spec 011 pricing slice (Phase 3, low priority). Logic drafted; depends on Phase 3 publish completion. Target 2026-08-10.

SRS-CAT-12: INVENTORY SYNC (STOCK LEVELS TO ETSY, PHASE 4)

Property	Detail
Statement	After fulfillment, stock levels in Odoo warehouse decrease (via standard picking/delivery flow). System shall periodically (every 2 hours) sync inventory quantity to Etsy API `/inventory` endpoint, per SKU. Updates only active listings. Handles backorder/low-stock thresholds (configurable per category). Log in etsy.api.log.
Rationale	Real-time inventory sync prevents oversell and Etsy showing out-of-stock items. Low-stock alerts enable rapid reorder.
Origin Spec(s)	Spec 004 P-HUB (inventory sync Phase 4), Spec 012 (inventory Phase 4)
Implementing Module + Model	`multichannel_hub_core` / cron job `_cron_sync_inventory_to_etsy()` (every 2 hours); `etsy.api.log` (inventory sync events)
Status	**Planned** — Phase 4 slice (inventory sync, low priority). Cron job scheduled (ir_cron_data.xml, commented, awaiting P4 dispatch). Tracker: `.claude/plans/006-master-plan-tracking.md` (Phase 4 = 75% complete; inventory = 0% within Phase 4). Target 2026-08-15.

SRS-CAT-13: ORIGINAL DESIGN DOCUMENT MANAGEMENT (ESTY-250)

Property	Detail
Statement	Product manager can upload and manage original design source files (AI/PSD/PDF mockups) linked to each product via a dedicated “Original Design” tab on the product form. Files are stored as product.document records tagged with `x_is_original_design=True`, distinct from the generic Documents smart button. File size is capped per configuration (default 10 MB); oversized uploads are rejected. Files are indexed and ordered by sequence for gallery display.
Rationale	Centralized design file management enables production team to access source artwork without manual file exchange; separates design assets from general product attachments.
Origin Spec(s)	ESTY-250 (product design document tab)
Implementing Module + Model	`multichannel_hub_core` / `product.document` (extension with `x_is_original_design` Boolean field, `_check_original_design_size_cap` constraint); `product.template` (field `x_original_design_ids` One2many with domain filter)
Status	**Shipped** — Extension field `x_is_original_design` (models/product_document.py line 35, Boolean, indexed). Size constraint `_check_original_design_size_cap` (line 43, @api.constrains) enforces threshold via `multichannel_hub.large_file_threshold_bytes` config parameter (default 10 MB). Product template relation `x_original_design_ids` (models/product_template.py line 75, One2many with domain `x_is_original_design=True`). Form tab “Original Design” (views/product_template_views.xml line 83, editable tree with drag-to-reorder sequence, file upload via datas field). Tests: test_phase1_original_design_db.py (column existence, field registration); test_phase2_original_design_orm.py (CRUD, tagging, domain filtering, size cap rejection, sequence ordering, O2M defaults).

Summary Table

Req ID	Title	Status	Module	Model	Tracker Reference
SRS-CAT-01	Product Hub & Channel Applicability	Shipped	multichannel_hub_core	product.product	Spec 004 P-HUB
SRS-CAT-02	SKU Auto-Derivation	Shipped	multichannel_hub_core	product.product, sku.derivation.rule	Spec 004 P-HUB-SKU-AUTODERIVE
SRS-CAT-03	Image Gallery & Routing	Shipped	multichannel_hub_core	product.template.image	Spec 004 P-HUB
SRS-CAT-04	Bulk Catalog Import	Shipped	multichannel_hub_core	multichannel.product.import.wizard	Spec 004 P-HUB
SRS-CAT-05	Listing Model & Intent	Shipped	multichannel_hub_core	multichannel.listing	Spec 004 P-LIST-MODEL
SRS-CAT-06	Channel Overrides	Shipped	multichannel_hub_core	multichannel.listing	Spec 011
SRS-CAT-07	Listing Publish (Odoo→Etsy)	Shipped	etsy_integration	etsy_listing_publisher service, multichannel.listing	MF-E2E-1 ×2 PASS (2026-07-04)
SRS-CAT-08	Listing Update (PATCH)	Planned	multichannel_hub_core	multichannel.listing	P3-LIST-04 (2026-07-25)
SRS-CAT-09	Listing Unpublish/Archive	Planned	multichannel_hub_core	multichannel.listing	P3-LIST-05 (2026-08-01)
SRS-CAT-10	Listing Sync (Inbound from Etsy)	Planned	multichannel_hub_core	multichannel.listing	P3-LIST-02b (2026-08-05)
SRS-CAT-11	Multi-Currency Pricing	Planned	multichannel_hub_core	multichannel.listing	Phase 3 (2026-08-10)
SRS-CAT-12	Inventory Sync to Etsy	Planned	multichannel_hub_core	etsy.api.log	Phase 4 (2026-08-15)
SRS-CAT-13	Original Design Document Tab	Shipped	multichannel_hub_core	product.document, product.template	ESTY-250

Document Version: 1.0

Next Section: 05-operations-admin.md — Dashboards, Health Monitoring, Logs, Configuration

Operations, Administration, and Observability Requirements

Title: Dashboards, Health Monitoring, API Logging, Configuration, Import Wizards, and Security Roles
Date: 2026-07-03
Status: Draft-for-owner-review
Source: Specs 003, 004, 013, 014, master plan Phase 1 (P1-01, P1-03, P1-08, etc.).

Scope

This section covers:

- Unified order, tracking, and operations dashboards (Phase 1)
- Sync health monitoring and observability
- API request/response logging
- System configuration (cron intervals, rate limits, thresholds)
- Import/export wizards (orders, products, tracking)
- Data migration tools
- ACL matrix and user group permissions
- I18n and Vietnamese business docs

Requirements (SRS-OPS-01 through SRS-OPS-16)

SRS-OPS-01: ORDER DASHBOARD & ANALYTICS

Property	Detail
Statement	Operations Manager shall access unified Order Dashboard showing: all orders grouped by shop and fulfillment status (pipeline stage). Pivot view shows order count/revenue by stage. List view displays order summary (order ID, customer, pipeline stage, design count, tracking status, due date). Filters: shop, date range, pipeline stage, customer. Color-coded decorations: red if overdue_approval, orange if stuck_route_badge. Export to Excel.
Rationale	Single pane of glass for order lifecycle visibility; enables rapid problem detection and SLA enforcement.
Origin Spec(s)	Spec 003 (tracking dashboard), Spec 004 P1-01 (order dashboard)
Implementing Module + Model	`multichannel_hub_core` / view `operations_dashboard_views.xml` (merged unified dashboard, see SRS-OPS-03); computed fields on `sale.order` (`is_overdue_approval`, `stuck_route_badge`)
Status	**Shipped (merged into SRS-OPS-03)** — Delivered inside the unified Operations Dashboard (`views/operations_dashboard_views.xml`) per CEO directive 2026-05-03, not as a separate view file. List view with pipeline-stage column, saved filters (`data/operations_dashboard_saved_filters.xml`), decorations (red = `is_overdue_approval`, orange = `stuck_route_badge`). Export via standard Odoo list export. Tests: `test_order_dashboard.py`. **Not delivered as originally specced:** separate pivot view, bus.bus live update, dedicated export server action — if still wanted, belongs in the spec 015 backlog.

SRS-OPS-02: TRACKING DASHBOARD & REAL-TIME SYNC

Property	Detail
Statement	Tracking Dashboard displays all orders with tracking numbers, grouped by carrier and delivery status. Real-time updates via bus.bus when fulfillment.tracking_number or tracking_state changes. Shows: carrier logo (from shipping.carrier icon field), tracking number (clickable deeplink via tracking_url), estimated delivery date, status badge (Label Requested, Shipped, Delivered). Filter by carrier, date range, state. Alert if tracking not updated > 7 days (stuck).
Rationale	Ops visibility into shipment pipeline; bus.bus enables instant customer query resolution. Stuck alerts catch fulfillment gaps.
Origin Spec(s)	Spec 003 P1-03 (tracking dashboard), Spec 004 P1-03 (fulfillment status)
Implementing Module + Model	`multichannel_hub_core` / view `operations_dashboard_views.xml` (merged unified dashboard, see SRS-OPS-03); `sale.order.fulfillment` (fields: `tracking_number`, `tracking_url`, `shipping_carrier_id`, `tracking_state`)
Status	**Shipped (merged into SRS-OPS-03)** — Delivered as tracking columns inside the unified Operations Dashboard list view (`views/operations_dashboard_views.xml`): `tracking_number`, `tracking_state` badge with per-state decorations (delivered/shipped/in_transit/returned). Tests: `test_tracking_dashboard.py`, `test_tracking_dashboard_db.py`. Tracking updates arrive asynchronously via Gearment webhook and tracking-import crons (5–15 min), **not** real-time bus.bus (no `_send_tracking_update_to_bus` exists). **Not delivered as originally specced:** kanban grouped by tracking_state, carrier icon display, bus.bus broadcast, dedicated stuck-alert (>7 days) decoration — spec 015 backlog candidates.

SRS-OPS-03: OPERATIONS DASHBOARD (CEO DIRECTIVE, UNIFIED)

Property	Detail
Statement	Single unified Operations Dashboard combining order + tracking + inventory + design status. CEO directive 2026-05-03: one dashboard, not split. Shows: KPI tiles (today's orders, ready-to-ship count, overdue count, low-stock SKUs). Grid of status cards (VN internal production queue, Gearment processing, customer feedback pending). Quick-action buttons (Mark Picked, Request Quote, Approve Design, Export Tracking). Drill-down to detailed reports (list views).
Rationale	Unified view prevents ops context switching; KPI tiles enable rapid SLA assessment. One source of truth for exec reporting.
Origin Spec(s)	Spec 004 P1-DASH-MERGE (CEO directive 2026-05-03), Phase 1 P1-01 exit criterion
Implementing Module + Model	`multichannel_hub_core` / view `operations_dashboard_views.xml` (decorated list-view cockpit); computed fields on `sale.order` / `sale.order.line` (`is_overdue_approval`, `stuck_route_badge`, `tracking_state` related)
Status	**Shipped** — Unified dashboard (`views/operations_dashboard_views.xml`): list + search views over order lines with pipeline, design, shipping-service, and tracking columns; saved filters (`data/operations_dashboard_saved_filters.xml`); bulk "Mark Shipped" server action (`action_server_bulk_mark_shipped`). Delivered as a decorated list-view cockpit, **not** a KPI-tile widget layout — the KPI tile fields named in earlier drafts (today_order_count, ready_to_ship_count, low_stock_sku_count) were never implemented. Tests: `test_operations_dashboard_db.py`, `test_operations_dashboard_orm.py`, `test_operations_dashboard_line_*.py`, `test_dashboards_db.py`.

SRS-OPS-04: SYNC HEALTH MONITORING & MULTI-CHANNEL OBSERVABILITY

Property	Detail
Statement	System shall track sync health for each data integration (Etsy API order fetch, email ingest, GKE tracking import, Gearment webhook, inventory sync). multichannel.sync.health model stores: last_sync_at, last_error (error message), consecutive_failures (counter), status (healthy
Rationale	Observability into sync pipeline; early alert on integration failures prevents data gaps. Multi-channel sync status enables triage.
Origin Spec(s)	Spec 004 P-HEALTH (sync health monitoring), Spec 003 (tracking dashboard)
Implementing Module + Model	`multichannel_hub_core` / `multichannel.sync.health` (fields: `channel_id` M2O, `metric_key` Char, `status` Selection, `last_run_at` / `last_success_at` / `last_error_at` Datetime, `last_error_message` Text, `rows_processed` / `rows_failed` Integer; `record()` upsert classmethod). Etsy channel additionally has its own `etsy.sync.health` (`etsy_integration/models/etsy_sync_health.py`).
Status	**Shipped (simpler shape than specced)** — Sync health models (`models/multichannel_sync_health.py` , `etsy_integration/models/etsy_sync_health.py`) record per-channel/per-metric checkpoints via `record()` , written by syncers and crons. Views: `views/multichannel_sync_health_views.xml` , `etsy_integration/views/etsy_sync_health_views.xml` . Tests: `test_phase1_sync_health_db.py` , `test_phase2_sync_health_orm.py` . **Not implemented as originally specced:** consecutive-failure counter, recovery_action field, threshold-based alert creation (no `_maybe_create_alert`), dedicated dashboard widget — status reflects last run only. Alerting escalation = spec 015 backlog candidate.

SRS-OPS-05: API REQUEST & RESPONSE LOGGING (ETSY.API.LOG / GEARMENT.API.LOG)

Property	Detail
Statement	External API calls to Etsy and Gearment shall be logged in per-channel log models (`etsy.api.log` , `gearment.api.log`) with endpoint, request/response payloads, status, and timestamps. Logs purged via retention crons. Used for debugging sync failures, API quota monitoring, and audit trail. GDrive/Gmail/ECB calls surface via sync-health checkpoints instead.
Rationale	API logs enable troubleshooting of integration failures; audit trail for compliance; quota monitoring prevents rate-limit surprises.
Origin Spec(s)	Spec 004 P-HEALTH (observability), Spec 013 (audit trail)
Implementing Module + Model	Per-channel log models, not one generic model: `etsy.api.log` (`etsy_integration/models/etsy_api_log.py`) and `gearment.api.log` (`multichannel_hub_fulfillment/models/gearment_api_log.py`)
Status	**Shipped (per-channel shape)** — Etsy calls logged to `etsy.api.log` (written by `etsy_api_client.py` / syncers); Gearment calls logged to `gearment.api.log` (written by the Gearment adapter/webhook dispatcher). Retention sweep cron `cron_etsy_api_log_cleanup` (`etsy_integration/data/ir_cron_data.xml`). There is **no** unified `multichannel.api.log` model; GDrive/Gmail/ECB calls are logged via sync-health checkpoints and `_logger` , not a dedicated API-log table.

SRS-OPS-06: SYSTEM CONFIGURATION PARAMETERS (IR.CONFIG_PARAMETER)

Property	Detail
Statement	System-wide settings shall be configurable via res.config.settings form (Odoo settings page). Parameters include: etsy_api_cron_interval (10 min default), email_ingest_cron_interval (10 min), gke_tracking_cron_interval (4 hours), gearment_quote_auto_fetch (Boolean), inventory_sync_interval (2 hours), smtp_rate_limit_per_minute (default 60), api_request_timeout_sec (default 30), gke_tracking_gdrive_folder_id, gearment_webhook_secret, ecb_currency_api_key. Form includes Help text explaining each parameter.
Rationale	Centralized config reduces code changes; enables ops tuning without deploy.
Origin Spec(s)	Spec 013 (system configuration), best practice
Implementing Module + Model	`etsy_integration` + `design` / `res.config.settings` extensions (`models/res_config_settings.py` + `views/res_config_settings_views.xml` in each); remaining knobs as raw `ir.config_parameter` keys (e.g. `multichannel_hub.large_file_threshold_bytes`, design auto-create toggle)
Status	**Shipped (subset)** — Settings-form extensions exist in `etsy_integration` and `design` (not `multichannel_hub_core`); hub/fulfillment tunables live as documented `ir.config_parameter` keys read at runtime. Cron intervals are tuned directly on the `ir.cron` records rather than via settings fields. Parameter set differs from the original draft list — see each module's `res_config_settings.py` for the authoritative fields.

SRS-OPS-07: CRON JOB MANAGEMENT & MONITORING

Property	Detail
Statement	System defines 10+ cron jobs (ir.cron records): order ingest (Etsy API + email), tracking import (GKE), tracking push (Etsy), Gearment webhook retry, currency rate sync, health monitor, api log purge, duplicate buyer recompute, overdue approval recompute, design file archival (deferred). Each cron has: name, model, method, interval_number, interval_type (minutes
Rationale	Centralized cron management enables ops to tune automation without code changes. Last-run dashboard enables quick diagnosis of sync gaps.
Origin Spec(s)	Spec 004 P-HEALTH (observability), best practice
Implementing Module + Model	`ir.cron` (Odoo core model), seeded across module data files: `etsy_integration/data/ir_cron_data.xml` + `ir_cron_currency_rates.xml` , `multichannel_hub_core/data/ir_cron_data.xml` + `product_catalog_cron.xml` , `multichannel_hub_fulfillment/data/logistics_partner_data.xml`
Status	**Shipped (standard tooling, no custom dashboard)** — 10+ cron jobs seeded across the module data files above (order sync 5 min, tracking push 5 min, email fetch 10 min, currency refresh, catalog sync, logistics inbox poll 15 min, API-log retention sweep, etc.). Enable/disable + last-run/next-run monitoring via standard Odoo Settings → Technical → Scheduled Actions. Failures surface through the sync-health checkpoints (SRS-OPS-04) and cron error logs. **Not implemented:** custom cron-monitor dashboard view (`cron_monitor_dashboard.xml` never existed).

SRS-OPS-08: ORDER IMPORT WIZARD (HISTORICAL + BULK)

Property	Detail
Statement	Wizard `import_orders_wizard.py` enables bulk import of historical orders (e.g., 17,659 from Google Sheets export). User uploads Excel file, system parses, creates sale.order + partner + product records in batch. Duplicates by etsy_transaction_id are skipped. Progress bar shows rows processed. Import log shows row-by-row status. Max batch size: 1000 rows per transaction (commits every 100).
Rationale	Bulk import enables historical data migration; batch size optimization prevents transaction bloat.
Origin Spec(s)	Spec 001 US2 (historical import 17,659 orders), Spec 013 (data migration)
Implementing Module + Model	`etsy_integration` / wizard `etsy.import.orders.wizard` (`wizards/import_orders_wizard.py` , transient)
Status	**Shipped** — Import wizard (`wizards/import_orders_wizard.py`): file upload, header mapping, row parsing, duplicate skip by transaction id, row-level issues recorded in the wizard's `validation_notes` (there is **no** separate `etsy.order.import.log` model). Tests: `test_import_wizard.py` , `test_import_wizard_headers.py` . Historical backlog remediation handled separately by `wizards/data_migration_wizard.py` (see SRS-OPS-11).

SRS-OPS-09: TRACKING NUMBER EXPORT WIZARD

Property	Detail
Statement	Wizard enables ops to export shipment manifest (tracking numbers, carriers, dates) to Excel for external reporting or carrier reconciliation. Wizard: date range picker, carrier filter, status filter. Output: Excel file with columns: order_id, customer_email, tracking_number, carrier, shipping_date, estimated_delivery, status. File can be downloaded or saved to GDrive.
Rationale	Export enables external reconciliation with shipping partners; centralized manifest.
Origin Spec(s)	Spec 003 (tracking dashboard), best practice
Implementing Module + Model	(planned) `multichannel_hub_core` / wizard `export_tracking_wizard.py` (transient); Excel generation via openpyxl
Status	**Planned** — No `export_tracking_wizard.py` exists in any module. Interim: standard Odoo list export from the unified Operations Dashboard (tracking columns are exportable). Dedicated wizard with GDrive save = spec 015 backlog.

SRS-OPS-10: PRODUCT BULK EXPORT

Property	Detail
Statement	Product manager can export product catalog to Excel for external use (e.g., sync to Google Sheets, PPC campaigns, analytics). Wizard: channel filter, date range (modified_since), include_images Boolean. Output: Excel with columns: sku, name, description, category, price, image_url, channel_applicability, in_stock, last_updated. Supports large exports (1000+ products).
Rationale	Centralized export enables syncing catalog to external systems without manual copying.
Origin Spec(s)	Spec 012 (inventory sync Phase 4), best practice
Implementing Module + Model	(planned) `multichannel_hub_core` / wizard `export_products_wizard.py` (transient)
Status	**Planned** — No `export_products_wizard.py` exists in any module. Interim: standard Odoo list export on product views. Dedicated catalog-export wizard = spec 015 backlog.

SRS-OPS-11: DATA MIGRATION & CLEANUP TOOLS

Property	Detail
Statement	Admin tools for data hygiene: (1) Duplicate order deduplication (by etsy_transaction_id): marks later duplicate as cancelled, merges notes. (2) Orphan design file cleanup: delete design files with no order_id + no route. (3) Stale Gearment quote purge: delete quotes > 60 days old with no PO. (4) Sync health reset: clear consecutive_failures counter. All tools have dry-run mode (preview impact, no commit).
Rationale	Data hygiene prevents bloat; dry-run mode enables safe testing before committing destructive operations.
Origin Spec(s)	Spec 013 (data cleanup), Phase 2 maintenance
Implementing Module + Model	`etsy_integration` / wizard `wizards/data_migration_wizard.py` (historical Etsy backlog remediation)
Status	**Partial (different shape than specced)** — The shipped tool is `etsy_integration/wizards/data_migration_wizard.py`: batched remediation of the historical order backlog with resumable iteration (`last_processed_id`), anomaly quarantine, per-batch `etsy.sync.health` checkpoints, fix helpers (financial config, shipping lines, prices-from-Excel, product config, confirm-and-complete), duplicate report generation + `action_apply_merges`. The four separately-named cleanup wizards from earlier drafts (`data_deduplication_wizard` , `design_cleanup_wizard` , `gearment_quote_purge_wizard` , `sync_health_reset_wizard`) do **not** exist. Remaining hygiene tools = spec 015 backlog.

SRS-OPS-12: ACL & RECORD RULE MATRIX

Property	Detail
Statement	Access control via ir.model.access.csv (model-level CRUD) + record rules (row-level filtering). Defined by user group (5 primary roles: Ops Lead, Production Lead, Gearment Operator, Etsy Manager, Product Lead). Example: Etsy Manager can CREATE/WRITE etsy.shop (own shop only), READ all orders, WRITE only order.fulfillment (approval). Production Lead can READ design.file (all), WRITE design.file.route (internal routes only), WRITE sale.order.fulfillment (VN internal pipeline only). ACL matrix documented in security/ir.model.access.csv.
Rationale	Least-privilege access prevents unauthorized data access. Record rules enable multi-tenant isolation (shop-level).
Origin Spec(s)	Spec 013 (security + audit), Spec 004 P1-08 (ACL skeleton)
Implementing Module + Model	All modules / `security/ir.model.access.csv` (~129 rows across the 4 modules: etsy_integration 44, multichannel_hub_core 65, multichannel_hub_fulfillment 16, design 4); record rules in each module's `security/*.xml`
Status	**Shipped** — ACL matrix defined across all four modules' `security/ir.model.access.csv`. Record rules include Etsy shop-scoped order access (`sale_order_etsy_shop_user_scope_rule`) and published-listing unlink protection (`rule_multichannel_listing_published_no_unlink`). Access enforcement is covered by scattered per-feature security assertions inside the module test suites rather than a single `test_acl_matrix.py` (which does not exist).

SRS-OPS-13: VIETNAMESE BUSINESS DOCUMENTATION (OWNER-FACING)

Property	Detail
Statement	All user-facing docs shall be in Vietnamese (owner preference). Includes: (1) BRD + SRS in Vietnamese (docs/owner/BRD_VN.md, docs/owner/SRS_VN.md). (2) Step-by-step how-to guides (docs/owner/HUONG_DAN_TAO_SAN_PHAM_VN.md, etc.). (3) Business flow diagrams (docs/owner/FLOW_DON_HANG_ETSY_VN.md, etc.). (4) UAT results + readiness checklists in Vietnamese. All docs auto-sync to Confluence space HEP via `.githooks/post-commit`.
Rationale	Owner understands operations better in native language; Confluence enables team collaboration.
Origin Spec(s)	Project requirement (owner = Vietnamese-speaking), Phase 1 P1-07 (i18n skeleton)
Implementing Module + Model	Documentation-only; docs/owner/*.md (22 files) + business-flow HTML pages (docs/owner/business-flows/); Confluence sync via `.githooks/post-commit`
Status	**Shipped** — 22 Vietnamese business docs in docs/owner/ plus the business-flows HTML set (v2). Flow guides and UAT material current. Confluence sync wired (post-commit hook, space HEP).

SRS-OPS-14: I18N UI SKELETON (PHASE 1 P1-07)

Property	Detail
Statement	System UI shall support i18n. Form labels, button text, menu items, and error messages shall use ir_translation (Odoo translation framework). Vietnamese translation (vi_VN) provided for Phase 1 UI (order dashboard, tracking dashboard, operations dashboard, fulfillment forms). Future: Amazon + other channel UIs to be translated.
Rationale	i18n enables Vietnamese ops team to use system in native language.
Origin Spec(s)	Spec 004 P1-07 (i18n skeleton), Phase 1 exit criterion
Implementing Module + Model	(planned) All modules / translation files (i18n/vi_VN.po) via the standard Odoo translation framework
Status	**Planned (not started)** — No `i18n/*.po` files exist in any custom module; no translation strings have been extracted. Some owner-facing surfaces (e.g. the `design` module's state labels) currently ship Vietnamese-first hardcoded strings instead of using the translation framework. Phase 1 P1-07 remains TODO in `.claude/plans/006-master-plan-tracking.md`.

SRS-OPS-15: AUDIT TRAIL & COMPLIANCE LOGGING

Property	Detail
Statement	Critical operations (order state transition, address change approval, design file routing, Gearment quote acceptance) shall be logged with: timestamp, user_id, action, before_state, after_state, change_reason. Logs stored in order.pipeline.transition.log (for state changes), audit.log (generic). Retained indefinitely. Ops can view audit trail via smart button on order + fulfillment records.
Rationale	Compliance audit trail enables traceability; investigation of disputes/issues.
Origin Spec(s)	Spec 013 (audit trail + compliance), Spec 004 P1-04 (address change approval audit)
Implementing Module + Model	`multichannel_hub_core` / `order.pipeline.transition.log` (order state audit; O2M `pipeline_transition_log_ids` on `sale.order`); domain-specific audit models: `etsy.address.change.request`, `etsy.shop.source.change.log` (etsy_integration)
Status	**Shipped (per-domain shape)** — Pipeline state changes write an `order.pipeline.transition.log` row in the same transaction (`models/order_pipeline_transition_log.py`; `sale.order.pipeline_transition_log_ids` O2M surfaces them on the order's Pipeline tab). Address changes audited in `etsy.address.change.request`; shop source toggles in `etsy.shop.source.change.log`. Chatter/tracking covers remaining field-level history. Tests: `test_audit_chatter_orm.py`, `test_audit_coverage_db.py`. There is **no** generic `audit.log` model — auditing is per-domain by design.

SRS-OPS-16: WEBHOOK RECEIVER ENDPOINT FOR GEARMENT (API SECURITY)

Property	Detail
Statement	System exposes REST endpoint `/gearment/webhook` (POST) to receive webhook notifications from Gearment. Endpoint validates HMAC-SHA256 signature (`X-Connect-Signature`, per Gearment API spec: url_path + nonce + timestamp + base64url(body)). Rejects unsigned/invalid requests. Audit-logs webhook events to `gearment.api.log`.
Rationale	Secure webhook endpoint prevents unauthorized state changes.
Origin Spec(s)	Spec 010 (Gearment dropship), reference_gearment_webhook_signature.md
Implementing Module + Model	`multichannel_hub_fulfillment` / controller `GearmentWebhookController` (`controllers/gearment_webhook.py`, POST `/gearment/webhook`, `auth='public'`, `csrf=False`); HMAC verification + event dispatch via the webhook dispatcher service
Status	**Shipped** — Webhook controller (`controllers/gearment_webhook.py`) with HMAC-SHA256 verification and per-request audit logging to `gearment.api.log`. Event routing via the dispatcher service. Tests: `test_webhook_discovery_db.py` / `_orm.py`, `test_webhook_dispatcher_db.py` / `_orm.py`. **Not implemented:** request rate limiting (deliberate — invalid signatures are logged and rejected; no rate-limit layer). **Note:** Webhook must be configured in Gearment dashboard.

Summary Table

Req ID	Title	Status	Module	Model	Tracker Reference
SRS-OPS-01	Order Dashboard	Shipped (merged into OPS-03)	multichannel_hub_core	sale.order	Spec 003, Spec 004 P1-01
SRS-OPS-02	Tracking Dashboard	Shipped (merged into OPS-03)	multichannel_hub_core	sale.order.fulfillment	Spec 003 P1-03, Spec 004
SRS-OPS-03	Operations Dashboard (Unified)	Shipped	multichannel_hub_core	sale.order / sale.order.line	Spec 004 P1-DASH-MERGE (CEO directive)
SRS-OPS-04	Sync Health Monitoring	Shipped	multichannel_hub_core + etsy_integration	multichannel.sync.health, etsy.sync.health	Spec 004 P-HEALTH
SRS-OPS-05	API Request/Response Logging	Shipped	etsy_integration + multichannel_hub_fulfillment	etsy.api.log, gearment.api.log	Spec 004 P-HEALTH
SRS-OPS-06	System Configuration	Shipped (subset)	etsy_integration + design	res.config.settings	Spec 013
SRS-OPS-07	Cron Job Management	Shipped	(all modules)	ir.cron	Spec 004 P-HEALTH
SRS-OPS-08	Order Import Wizard	Shipped	etsy_integration	etsy.import.orders.wizard	Spec 001 US2, Spec 013
SRS-OPS-09	Tracking Export Wizard	Planned	multichannel_hub_core	(export only)	Spec 003
SRS-OPS-10	Product Bulk Export	Planned	multichannel_hub_core	(export only)	Spec 012
SRS-OPS-11	Data Migration & Cleanup Tools	Partial	etsy_integration	data_migration_wizard	Phase 2 (2026-07-30)
SRS-OPS-12	ACL & Record Rule Matrix	Shipped	(all modules)	ir.model.access	Spec 013, Spec 004 P1-08
SRS-OPS-13	Vietnamese Business Docs	Shipped	(documentation)	(docs/owner)	Phase 1 P1-07
SRS-OPS-14	I18n UI Skeleton	Planned	(all modules)	ir_translation	Phase 1 P1-07 (2026-07-20)
SRS-OPS-	Audit Trail & Compliance	Shipped	multichannel_hub_core + etsy_integration	order.pipeline.transition.log + per-domain audit models	Spec 013, Spec 004

Req ID	Title	Status	Module	Model	Tracker Reference
15					
SRS-OPS-16	Webhook Receiver (Gearment)	Shipped	multichannel_hub_fulfillment	(controller /gearment/webhook)	Spec 010

Cross-Module Summary

Total Requirements: 70 (14 Etsy + 18 Order/Fulfillment + 12 Catalog + 16 Operations)

Status Breakdown (updated 2026-07-04 after code-drift audit):

- **Shipped:** 56 requirements (80%)
- **Partial:** 4 requirements (6%) — Address Change Approval UI, Design Auto-Archive, Listing Publish (Phase 3), Data Migration/Cleanup Tools
- **Planned:** 10 requirements (14%) — Listing Fetch/Publish/Update/Unpublish/Inbound-Sync (Phase 3), Inventory Sync (Phase 4), Tracking Export Wizard, Product Bulk Export, I18n UI Skeleton

Phase Mapping:

- **Phase 0 (Cleanup + Sandbox):** 27 tasks, 89% complete ✅
- **Phase 1 (Dashboards + Approvals):** 55 tasks, 78% complete 🟨
- **Phase 2 (Tracking + GDrive):** 8 tasks, 75% complete 🟨
- **Phase 3 (Catalog Publish):** 73 tasks, 1% complete 🔴 (CRITICAL PATH)
- **Phase 4 (Gearment + Returns):** 8 tasks, 75% complete 🟨
- **Phase 5 (Amazon + Inventory):** 5 tasks, 0% complete 🔴

Document Version: 1.0
Last Section

Appendix: Key ADRs Referenced

ADR	Title	Impact on Requirements
ADR-003	Four-module decomposition	Foundation for all requirements (etsy_integration, design, multichannel_hub_core, multichannel_hub_fulfillment)
ADR-007	Fulfillment delegation mixin	SRS-ORD-18 (sale.order.fulfillment separation)
ADR-008	API-first pivot	SRS-ETSY-04 (primary order ingest), SRS-ETSY-03/05 (email fallback)
ADR-010	Hybrid dropship + MTO	SRS-ORD-01 (three pipelines), SRS-CAT-01 (channel applicability)
ADR-014	Phase 3 priority (2026-05-23)	SRS-CAT-07/08/09/10 (listing publish, Phase 3 critical path)

Document Version: 1.0
Prepared By: Claude Code (consolidated from module reports + evidence base)
Date: 2026-07-03

Status: Draft-for-owner-review